尚硅谷嵌入式项目之 AI 聊天机器人

（作者：尚硅谷研究院）

版本：V1.0

第 1 章 项目概述

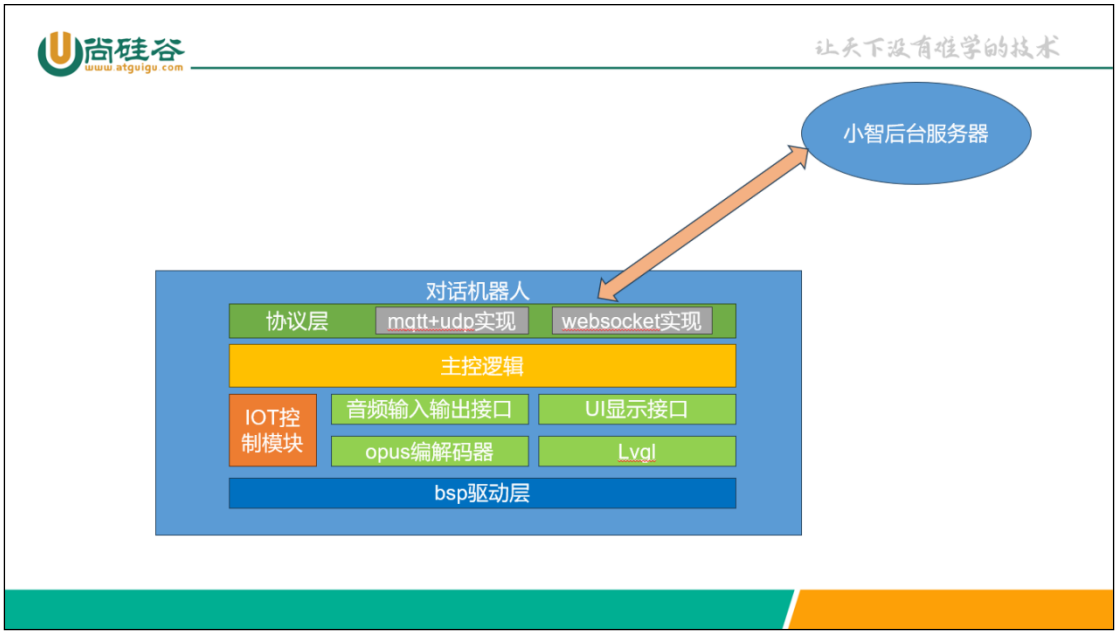
1.1 项目背景

随着大模型的火热，自然会话 app 和设备风靡一时。在嵌入式领域，小智智能对话机器人是最流行的。小智智能机器人项目分嵌入式前端和网站后端。嵌入式前端原本是使用 C++ 实现的，本项目是使用 C 语言重新实现小智项目，使用小智通信协议完成和小智服务器交互的对话机器人。

1.2 项目目标

本项目主要是使用 esp32 对小智通信协议的 C 语言重新实现，同时将所有依赖的下游 components 替换为 espressif 官方库（官方库一般有更好的维护周期和技术支持）。

1.3 项目架构



1）协议层

小智协议使用 Json 和 opus 音频数据和服务器交互，有两种实现：

- mqtt + udp。这种协议使用 mqtt 发送和接收 json 数据，使用 upd 发送和接收 opus 音频数据流。
- websocket。这种协议直接使用 websocket 协议处理一切数据。

目前本项目只实现了 websocket 协议。

2）主控逻辑

主控逻辑主要负责聊天机器人的状态切换：例如空闲状态、讲话状态、倾听状态、激活状态，升级状态等。

3）IOT 控制模块

这个模块主要实现后台大模型对于机器人前端的设备控制能力。例如可以通过对话控制大模型开启或者关闭设备的扬声器，调整扬声器音量等。

4）音频输入输出接口

音频输入输出接口主要负责唤醒识别，语音采集和播放服务器的音频数据。其底层依赖于 opus 编解码器和 esp-sr 语音识别模型。

5）UI 显示接口

UI 显示接口主要负责在 lcd 上显示大模型的响应，包括 llm 表情，使机器人更生动，其依赖于 lvgl 库。本项目目前还未实现 UI 显示部分。

6）bsp 驱动层

BSP 驱动层主要是对开发板的适配。如果需要将本项目移植到其他开发板，直接在 BSP 驱动层适配即可。

第 2 章 通信协议解析

2.1 OTA 版本检查和激活

程序初始化结束后首先要向服务器发送 post 请求

Header	
Device-Id	设备 WiFi Mac 地址
Client-Id	UUID，第一次随机生成，之后需要报存
Accept-Language	语言标志，我们直接选 zh-CN
User-Agent	开发板名称/固件版本

Body


```
{
"application":
    {
    "compile_time": "Apr 28 2025",
    "elf_sha256":
"c8ded1fda8e3716d3f5da1ebcf863a5d3bedcd8f273e6be9e9990fca377eb32c",
    "idf_version": "v5.3.2-dirty",
    "name": "template-app",
    "version": "7354bef-dirty"
    },
"board":
    {
    "channel": 1,
    "ip": "192.168.1.100",
    "mac": "f0:f5:bd:50:a9:44",
    "name": "bread-compact-wifi",
    "rssi": -50,
    "ssid": "atguigu",
    "type": "bread-compact-wifi"
    },
"chip_info":
    {
    "cores": 2,
    "features": 18,
    "model": 9,
    "revision": 2
    },
```

```
"chip_model_name": "esp32s3",
"flash_size": 8388608,
"mac_address": "f0:f5:bd:50:a9:44",
"minimum_free_heap_size": 8494904,
"ota":
    {
    "label": "ota_0"
    },
"partition_table":
    [
        {
        "address": 36864,
        "label": "nvs",
        "size": 16384,
        "subtype": 2,
        "type": 1
        },
        {
        "address": 53248,
        "label": "otadata",
        "size": 8192,
        "subtype": 0,
        "type": 1
        },
        {
        "address": 61440,
        "label": "phy_init",
        "size": 4096,
        "subtype": 1,
```

```
            "type": 1
            },
            {
            "address": 65536,
            "label": "model",
            "size": 983040,
            "subtype": 130,
            "type": 1
            },
            {
            "address": 1048576,
            "label": "ota_0",
            "size": 3670016,
            "subtype": 16,
            "type": 0
            },
            {
            "address": 4718592,
            "label": "ota_1",
            "size": 3670016,
            "subtype": 17,
            "type": 0
            }
        ],
"psram_size": 8388608,
"uuid": "3d3b749b-6a6a-4b33-ac88-3a5afe1c5a96",
"version": 2
}
```


Body 如上所示，需要根据开发板情况填写。服务器在收到我们的请求后，会回复：

Body


```
{
"activation":
    {
    "challenge": "97582491-9a14-42a9-a18d-dca62eb67e63",
    "code": "284596",
    "message": "xiaozhi.me\n284596"
    },
"firmware":
    {
    "url": "",
    "version": "7354bef-dirty"
    },
"mqtt":
    {
    "client_id": "",
    "endpoint": "",
    "password": "",
    "publish_topic": "",
    "subscribe_topic": "",
    "username": ""
    },
"server_time":
    {
    "timestamp": 1745824197471,
    "timezone_offset": 480
    }
}
```


其中包含几个重要部分：

（1）如果设备未激活，会发送激活信息；我们需要将激活信息显示在设备中

（2）如果设备版本过旧，会发送 OTA 信息（这部分我们屏蔽了，因为小智官方的固件不适配我们的门铃开发板）

（3）mqtt 协议需要的服务器相关信息。

（4）服务器时间戳以及时区

这部分代码在 OTA 模块中实现，是独立于具体协议的，不论使用什么协议都需要在启动时发送上面的数据包。

2.2 Websocket 通信协议

小智机器人对话时，需要和服务器建立流式通信信道，用来收发 opus 音频数据；同时还需要收发 json 格式的协议数据。官方服务器支持两种实现：websocket 协议和 MQTT+UDP 协议，下面我们以 websocket 协议讲解。

2.2.1 Websocket Header

Header	
Device-Id	设备 WiFi Mac 地址
Client-Id	UUID
Authorization	Bearer WEBSOCKET_TOKEN
Protocol-Version	1

2.2.2 建立信道

当设备被唤醒时（例如识别到唤醒语音），首先要向服务器发送建立信道的握手信息：

```
{
"audio_params":
    {
    "channels": 1,
    "format": "opus",
    "frame_duration": 60,
    "sample_rate": 16000
    },
"transport": "websocket",
"type": "hello",
"version": 1
}
```

其中需要包含一些服务器需要的信息：

（1）发送 opus 声音的声道数量

（2）编码格式（截止此刻官方服务器只支持 opus 编码）

（3）opus 帧长度

（4）采样率

服务器回复：

```
{
"audio_params":
    {
    "channels": 1,
    "format": "opus",
    "frame_duration": 60,
    "sample_rate": 24000
    },
"session_id": "e50e69eb",
"transport": "websocket",
"type": "hello",
"version": 1
}
```

其中包含服务器发回的 opus 片段采样率，声道等信息。这部分实现详见下面的 protocol 代码。

2.2.3 IOT 模块控制信息

建立信道后，设备需要向服务器发送本设备的 IOT 外设，以供 AI 调用。

```
{
"descriptors":
    [
        {
        "description": "Speaker",
        "methods":
            {
            "SetMute":
                {
                "description": "Set mute status",
                "parameters":
                    {
                    "mute":
                        {
                        "description": "Mute status",
                        "type": "boolean"
                        }
                    }
                },
            "SetVolume":
                {
                "description": "Set volume level",
                "parameters":
                    {
                    "volume":
                        {
                        "description": "Volume level[0-100]",
                        "type": "number"
                        }
                    }
                }
```

```
            },
        "name": "Speaker",
        "properties":
            {
            "mute":
                {
                "description": "Mute status",
                "type": "boolean"
                },
            "volume":
                {
                "description": "Volume level[0-100]",
                "type": "number"
                }
            }
        }
    ],
"session_id": "e50e69eb",
"type": "iot",
"update": true
}
```

还需要向服务器发送当前设备状态：

```
{
"session_id": "e50e69eb",
"states":
    [
        {
        "name": "Speaker",
        "state":
            {
            "mute": false,
            "volume": 60
            }
        }
    ],
"type": "iot",
"update": true
}
```

这两条信息服务器不会有任何回复。这部分代码使用 C 语言包装非常麻烦，详见 IOT 控制模块。同时我们还需要再 IOT 控制模块实现回调，当服务器发送类似下面的调用函数 JSON，我们需要调整设备相应的参数：

```
{
"commands":
    [
        {
        "method": "SetVolume",
        "name": "Speaker",
        "parameters":
            {
            "volume": 100
            }
        }
    ],
"session_id": "39e871fb",
```

```
"type": "iot"
}
```

2.2.4 对话

1）发送唤醒词

当设备唤醒后，我们首先需要向设备发送唤醒词：

```
{
"session_id": "e50e69eb",
"state": "detect",
"text": "你好小智",
"type": "listen"
}
```

此时服务器会回复我们语音和文字，具体内容在之后解析

2）发送监听请求

接下来我们需要向服务器发送监听请求，少了这一条服务器不会解析我们发过去的 opus 语音片段

```
{
"mode": "auto",
"session_id": "e50e69eb",
"state": "start",
"type": "listen"
}
```

3）发送 opus 片段

发送完监听请求后，就可以发送 opus 片段了。服务器端会使用 stt 识别声音片段，并给我们回复。发送声音片段时，websocket 包要标记位 binary。

2.2.5 服务器回复解析

当我们发送唤醒词或者 opus 片段后，服务器会给我们回复。回复的种类包括：

1）语音识别结果

我们发送到服务器的 opus 片段，服务器会做语音识别，并以 json 的形式发回来，我们可以在设备上显示：

```
{
"session_id": "e373d277",
"text": "今天天气怎么样",
"type": "stt"
}
```

2）大模型意图识别表情回复

大模型在回复我们时，会回复我们表情，我们可以根据这个表情在设备上显示相应动画，让机器人更生动

```
{
"emotion": "happy",
```

```
"session_id": "e373d277",
"text": "😊",
"type": "llm"
}
```

3）文字回复

文字回复共有三种状态，start，stop，sentence_start。

没次对话开始时先发送 start。每句话会发送一次 sentence_start。对话结束时发送 stop。

服务器会将大模型的回复以文字的形式发给我们，我们可以在设备上显示：

```
{
"session_id": "e373d277",
"state": " start",
"type": "tts"
}

{
"session_id": "e373d277",
"state": "sentence_start",
"text": "今天北京天气挺好的，晴朗，气温 21 度，西北风 5 级，",
"type": "tts"
}
{
"session_id": "e373d277",
"state": "stop",
"type": "tts"
}
```

4）opus 音频片段

大模型会将回复的音频片段以 binary 包的形式发回来。我们需要将所有 binary 包解码并播放。

2.2.6 结束对话

结束对话有两种形式，一种是我们主动向服务器发送结束对话的请求，一种是我们向机器人发送类似“退下”的指令，服务器端会根据大模型行为主动结束通信，并将协议信道关闭，此时一次对话结束。

1）主动结束请求

```
{
"session_id": "e373d277",
"state": "stop",
"type": "listen"
}
```

2.2.7 中断对话

当 AI 机器人正在讲话，我们可以发送中断对话的请求，此时机器人会停止说话，而我们可以继续发送监听请求进行下一轮对话

```
{
"reason": "wake_word_detected",
"session_id": "e373d277",
"type": "abort"
}
```

第 3 章 项目代码

3.1 构建 ESP 项目框架

使用 ESP 空项目模板构建 ESP 空项目，并添加依赖。依赖列表如下：

1）idf_component.yml

```
## IDF Component Manager Manifest File
dependencies:
  espressif/esp_codec_dev: ^1.3.4
  espressif/qrcode: ^0.1.0~2
  espressif/esp-sr: ^2.1.1
  espressif/esp_audio_codec: ^2.3.0
  espressif/esp_websocket_client: ^1.4.0
  espressif/led_strip: ^3.0.1
  ## Required IDF version
  idf:
    version: '>=5.3'
```

3.2 bsp 驱动层

BSP 驱动层需要实现硬件的必要设备句柄，例如音频输入输出句柄，LCD 显示句柄，WIFI 初始化，并实现所有开发板相关信息获取，例如 mac 地址，uuid 读写，获取设备信息 json 等。

3.2.1 BSP 配置

1）bsp_config.h

```
#ifndef BSP_CONFIG_H
#define BSP_CONFIG_H

#define BSP_CODEC_I2C_PORT 0
#define BSP_CODEC_I2C_SCL_PIN 1
#define BSP_CODEC_I2C_SDA_PIN 0

#define BSP_CODEC_I2S_PORT 0
#define BSP_CODEC_I2S_WS_PIN 5
#define BSP_CODEC_I2S_BCK_PIN 2
#define BSP_CODEC_I2S_MCK_PIN 3
#define BSP_CODEC_I2S_DI_PIN 4
```

```
#define BSP_CODEC_I2S_DO_PIN 6

#define BSP_CODEC_PA_PIN 7
#define BSP_CODEC_SAMPLE_RATE 16000
#define BSP_CODEC_CHANNEL_COUNT 1
#define BSP_CODEC_BIT_PER_SAMPLE 16

#define BSP_DISPLAY_ENABLE
#define BSP_DISPLAY_ST7789
#define BSP_DISPLAY_ST7789_SPI_PORT (1)
#define BSP_DISPLAY_ST7789_CS_PIN (21)
#define BSP_DISPLAY_ST7789_DC_PIN (45)
#define BSP_DISPLAY_ST7789_RST_PIN (16)
#define BSP_DISPLAY_BG_PIN (40)
#define BSP_DISPLAY_WIDTH (240)
#define BSP_DISPLAY_HEIGHT (320)

#define BSP_LED_PIN 46

#endif
```

3.2.2 音频初始化

1）bsp_codec.h

```
#ifndef BSP_CODEC_H
#define BSP_CODEC_H

#include <esp_types.h>

void bsp_codec_init(void);

void bsp_codec_read(void *buf, size_t len);

void bsp_codec_write(void *buf, size_t len);

void bsp_codec_set_mic_gain(float gain);

void bsp_codec_set_volume(int vol);

void bsp_codec_mute(bool mute);

#endif
```

2）bsp_codec.c

```
#include "bsp_codec.h"
```

```
#include "esp_codec_dev.h"
#include "esp_codec_dev_defaults.h"
#include <esp_log.h>
#include <esp_err.h>
#include <assert.h>
#include <driver/i2c.h>
#include <driver/i2s_std.h>
#include "bsp_config.h"

#define TAG "[BSP] Codec"

// 编码器句柄，通过这个指针，指向 es8311 的结构体
static esp_codec_dev_handle_t codec_dev = NULL;

static void bsp_i2s_init(i2s_chan_handle_t *tx_handle_p,
i2s_chan_handle_t *rx_handle_p)
{
    i2s_chan_config_t chan_cfg =
I2S_CHANNEL_DEFAULT_CONFIG(BSP_CODEC_I2S_PORT, I2S_ROLE_MASTER);
    chan_cfg.auto_clear = true; // Auto clear the legacy data in the
DMA buffer
    ESP_ERROR_CHECK(i2s_new_channel(&chan_cfg, tx_handle_p,
rx_handle_p));

    i2s_std_config_t std_cfg = {
        .clk_cfg = I2S_STD_CLK_DEFAULT_CONFIG(BSP_CODEC_SAMPLE_RATE),
        .slot_cfg =
I2S_STD_PHILIPS_SLOT_DEFAULT_CONFIG(BSP_CODEC_BIT_PER_SAMPLE,
BSP_CODEC_CHANNEL_COUNT),
        .gpio_cfg = {
            .mclk = BSP_CODEC_I2S_MCK_PIN,
            .bclk = BSP_CODEC_I2S_BCK_PIN,
            .ws = BSP_CODEC_I2S_WS_PIN,
            .dout = BSP_CODEC_I2S_DO_PIN,
            .din = BSP_CODEC_I2S_DI_PIN,
            .invert_flags = {
                .mclk_inv = false,
                .bclk_inv = false,
                .ws_inv = false,
            },
        },
    };
    std_cfg.clk_cfg.mclk_multiple = I2S_MCLK_MULTIPLE_128;
```

```
    ESP_ERROR_CHECK(i2s_channel_init_std_mode(*tx_handle_p, &std_cfg));
    ESP_ERROR_CHECK(i2s_channel_init_std_mode(*rx_handle_p, &std_cfg));
    ESP_ERROR_CHECK(i2s_channel_enable(*tx_handle_p));
    ESP_ERROR_CHECK(i2s_channel_enable(*rx_handle_p));
}

static void bsp_i2c_init(void)
{
    const i2c_config_t es_i2c_cfg = {
        .sda_io_num = BSP_CODEC_I2C_SDA_PIN,
        .scl_io_num = BSP_CODEC_I2C_SCL_PIN,
        .mode = I2C_MODE_MASTER,
        .sda_pullup_en = GPIO_PULLUP_ENABLE,
        .scl_pullup_en = GPIO_PULLUP_ENABLE,
        .master.clk_speed = 100000,
    };
    ESP_ERROR_CHECK(i2c_param_config(BSP_CODEC_I2C_PORT, &es_i2c_cfg));
    ESP_ERROR_CHECK(i2c_driver_install(BSP_CODEC_I2C_PORT,
I2C_MODE_MASTER, 0, 0, 0));
}

void bsp_codec_init(void)
{
    if (codec_dev != NULL)
    {
        ESP_LOGW(TAG, "Codec has been initialized");
        return;
    }

    i2s_chan_handle_t tx_handle = NULL;
    i2s_chan_handle_t rx_handle = NULL;

    bsp_i2s_init(&tx_handle, &rx_handle);
    // 构建数据接口
    audio_codec_i2s_cfg_t i2s_cfg = {
        .port = BSP_CODEC_I2S_PORT,
        .tx_handle = tx_handle,
        .rx_handle = rx_handle,
    };
    const audio_codec_data_if_t *data_if =
audio_codec_new_i2s_data(&i2s_cfg);

    bsp_i2c_init();
    audio_codec_i2c_cfg_t i2c_cfg = {
```

```
        .port = BSP_CODEC_I2C_PORT,
        .addr = ES8311_CODEC_DEFAULT_ADDR,
    };
    const audio_codec_ctrl_if_t *out_ctrl_if =
audio_codec_new_i2c_ctrl(&i2c_cfg);

    const audio_codec_gpio_if_t *gpio_if = audio_codec_new_gpio();

    es8311_codec_cfg_t es8311_cfg = {
        .codec_mode = ESP_CODEC_DEV_WORK_MODE_BOTH,
        .ctrl_if = out_ctrl_if,
        .gpio_if = gpio_if,
        .pa_pin = BSP_CODEC_PA_PIN,
        .use_mclk = true,
        .mclk_div = I2S_MCLK_MULTIPLE_128,
    };
    const audio_codec_if_t *out_codec_if =
es8311_codec_new(&es8311_cfg);

    esp_codec_dev_cfg_t codec_cfg = {
        .dev_type = ESP_CODEC_DEV_TYPE_IN_OUT,
        .codec_if = out_codec_if,
        .data_if = data_if,
    };

    codec_dev = esp_codec_dev_new(&codec_cfg);
    assert(codec_dev);

    esp_codec_dev_sample_info_t sample_info = {
        .bits_per_sample = BSP_CODEC_BIT_PER_SAMPLE,
        .channel = BSP_CODEC_CHANNEL_COUNT,
        .sample_rate = BSP_CODEC_SAMPLE_RATE,
        .channel_mask = ESP_CODEC_DEV_MAKE_CHANNEL_MASK(0),
        .mclk_multiple = I2S_MCLK_MULTIPLE_128,
    };

    ESP_ERROR_CHECK(esp_codec_dev_open(codec_dev, &sample_info));
}

void bsp_codec_read(void *buf, size_t len)
{
    int ret = esp_codec_dev_read(codec_dev, buf, len);
    switch (ret)
    {
```

```
    case ESP_CODEC_DEV_INVALID_ARG:
        ESP_LOGE(TAG, "Codec Read Error: Invalid Arguments");
        break;
    case ESP_CODEC_DEV_NOT_SUPPORT:
        ESP_LOGE(TAG, "Codec Read Error: Not Support");
        break;
    case ESP_CODEC_DEV_WRONG_STATE:
        ESP_LOGE(TAG, "Codec Read Error: Wrong State");
        break;
    default:
        break;
    }
}

void bsp_codec_write(void *buf, size_t len)
{
    ESP_ERROR_CHECK(esp_codec_dev_write(codec_dev, buf, len));
}

void bsp_codec_set_mic_gain(float gain)
{
    ESP_ERROR_CHECK(esp_codec_dev_set_in_gain(codec_dev, gain));
}

void bsp_codec_set_volume(int vol)
{
    ESP_ERROR_CHECK(esp_codec_dev_set_out_vol(codec_dev, vol));
}

void bsp_codec_mute(bool mute)
{
    ESP_ERROR_CHECK(esp_codec_dev_set_out_mute(codec_dev, mute));
}
```

3.2.3 LED 初始化

1）bsp_led.h

```
#ifndef BSP_LED_H
#define BSP_LED_H

#include "bsp_config.h"

typedef enum
{
```

```
    LED_STATE_STARTING,
    LED_STATE_IDLE,
    LED_STATE_ACTIVATING,
    LED_STATE_CONNECTING,
    LED_STATE_LISTENING,
    LED_STATE_SPEAKING,
} bsp_led_state_t;

void bsp_led_init(void);

void bsp_led_set_state(bsp_led_state_t state);

#endif
```

2）bsp_led.c

```
#include "bsp_led.h"
#include "led_strip.h"

#define LED_COLOR_GREEN 0, 7, 0
#define LED_COLOR_PURPLE 7, 0, 7
#define LED_COLOR_BLUE 0, 0, 7
#define LED_COLOR_ORANGE 7, 3, 0
#define LED_COLOR_CYAN 0, 7, 7
#define LED_COLOR_RED 7, 0, 0

static led_strip_handle_t s_led_strip = NULL;

void bsp_led_init(void)
{
    /// LED strip common configuration
    led_strip_config_t strip_config = {
        .strip_gpio_num = BSP_LED_PIN,                                  //
The GPIO that connected to the LED strip's data line
        .max_leds = 2,                                                  //
The number of LEDs in the strip,
        .led_model = LED_MODEL_WS2812,                                  //
LED strip model, it determines the bit timing
        .color_component_format = LED_STRIP_COLOR_COMPONENT_FMT_GRB, //
The color component format is G-R-B
        .flags = {
            .invert_out = false, // don't invert the output signal
        },
    };

    /// RMT backend specific configuration
```

```
    led_strip_rmt_config_t rmt_config = {
        .clk_src = RMT_CLK_SRC_DEFAULT,    // different clock source
can lead to different power consumption
        .resolution_hz = 10 * 1000 * 1000, // RMT counter clock
frequency: 10MHz
        .mem_block_symbols = 64,           // the memory size of each
RMT channel, in words (4 bytes)
        .flags = {
            .with_dma = true, // DMA feature is available on chips like
ESP32-S3/P4
        },
    };

    /// Create the LED strip object
    ESP_ERROR_CHECK(led_strip_new_rmt_device(&strip_config,
&rmt_config, &s_led_strip));
}

void bsp_led_set_state(bsp_led_state_t state)
{
    switch (state)
    {
    case LED_STATE_STARTING:
        led_strip_set_pixel(s_led_strip, 0, LED_COLOR_PURPLE);
        led_strip_set_pixel(s_led_strip, 1, LED_COLOR_PURPLE);
        led_strip_refresh(s_led_strip);
        break;
    case LED_STATE_IDLE:
        // 绿色
        led_strip_set_pixel(s_led_strip, 0, LED_COLOR_GREEN);
        led_strip_set_pixel(s_led_strip, 1, LED_COLOR_GREEN);
        led_strip_refresh(s_led_strip);
        break;
    case LED_STATE_ACTIVATING:
        led_strip_set_pixel(s_led_strip, 0, LED_COLOR_BLUE);
        led_strip_set_pixel(s_led_strip, 1, LED_COLOR_BLUE);
        led_strip_refresh(s_led_strip);
        break;
    case LED_STATE_CONNECTING:
        led_strip_set_pixel(s_led_strip, 0, LED_COLOR_ORANGE);
        led_strip_set_pixel(s_led_strip, 1, LED_COLOR_ORANGE);
        led_strip_refresh(s_led_strip);
        break;
    case LED_STATE_LISTENING:
```

```
        led_strip_set_pixel(s_led_strip, 0, LED_COLOR_CYAN);
        led_strip_set_pixel(s_led_strip, 1, LED_COLOR_CYAN);
        led_strip_refresh(s_led_strip);
        break;
    case LED_STATE_SPEAKING:
        led_strip_set_pixel(s_led_strip, 0, LED_COLOR_RED);
        led_strip_set_pixel(s_led_strip, 1, LED_COLOR_RED);
        led_strip_refresh(s_led_strip);
        break;
    default:
        break;
    }
}
```

3.2.4 WiFi 初始化

1）bsp_wifi.h

```
#ifndef BSP_WIFI_H
#define BSP_WIFI_H

void bsp_wifi_init(void);

#endif
```

2）bsp_wifi.c

```
#include "bsp_wifi.h"
#include <nvs_flash.h>
#include <freertos/FreeRTOS.h>
#include <freertos/event_groups.h>
#include <esp_wifi.h>
#include <esp_err.h>
#include <esp_log.h>

#include <wifi_provisioning/manager.h>
#include <wifi_provisioning/scheme_ble.h>

#include "qrcode.h"

#define TAG "[BSP] WiFi"
#define PROV_QR_VERSION         "v1"
#define PROV_TRANSPORT_BLE      "ble"
#define QRCODE_BASE_URL         "https://espressif.github.io/esp-
jumpstart/qrcode.html"

#define WIFI_CONNECTED_BIT (1 << 0)
```

```
static EventGroupHandle_t s_wifi_event_group = NULL;

static void bsp_nvs_flash_init(void)
{
    esp_err_t ret = nvs_flash_init();
    if (ret == ESP_ERR_NVS_NO_FREE_PAGES || ret ==
ESP_ERR_NVS_NEW_VERSION_FOUND)
    {
        ESP_ERROR_CHECK(nvs_flash_erase());
        ret = nvs_flash_init();
    }
    ESP_ERROR_CHECK(ret);
}

static void event_handler(void *arg, esp_event_base_t event_base,
                          int32_t event_id, void *event_data)
{
    if (event_base == WIFI_PROV_EVENT && event_id == WIFI_PROV_END)
    {
        // 配网结束，释放配网 Manager
        wifi_prov_mgr_deinit();
    }
    else if (event_base == WIFI_EVENT && event_id ==
WIFI_EVENT_STA_START)
    {
        esp_wifi_connect();
    }
    else if (event_base == WIFI_EVENT && event_id ==
WIFI_EVENT_STA_DISCONNECTED)
    {
        ESP_LOGW(TAG, "WiFi disconnected, reconnecting...");
        esp_wifi_connect();
    }
    else if (event_base == IP_EVENT && event_id == IP_EVENT_STA_GOT_IP)
    {
        xEventGroupSetBits(s_wifi_event_group, WIFI_CONNECTED_BIT);
    }
}

static void get_device_service_name(char *service_name, size_t max)
{
    uint8_t eth_mac[6];
    const char *ssid_prefix = "PROV_";
```

```
    esp_wifi_get_mac(WIFI_IF_STA, eth_mac);
    snprintf(service_name, max, "%s%02X%02X%02X",
             ssid_prefix, eth_mac[3], eth_mac[4], eth_mac[5]);
}

static void wifi_prov_print_qr(const char *name, const char *username,
const char *pop, const char *transport)
{
    if (!name || !transport) {
        ESP_LOGW(TAG, "Cannot generate QR code payload. Data
missing.");
        return;
    }
    char payload[150] = {0};
    if (pop) {
        snprintf(payload, sizeof(payload),
"{\"ver\":\"%s\",\"name\":\"%s\"" \
                    ",\"pop\":\"%s\",\"transport\":\"%s\"}",
                    PROV_QR_VERSION, name, pop, transport);
    } else {
        snprintf(payload, sizeof(payload),
"{\"ver\":\"%s\",\"name\":\"%s\"" \
                    ",\"transport\":\"%s\"}",
                    PROV_QR_VERSION, name, transport);
    }
    ESP_LOGI(TAG, "Scan this QR code from the provisioning application
for Provisioning.");
    esp_qrcode_config_t cfg = ESP_QRCODE_CONFIG_DEFAULT();
    esp_qrcode_generate(&cfg, payload);
    ESP_LOGI(TAG, "If QR code is not visible, copy paste the below URL
in a browser.\n%s?data=%s", QRCODE_BASE_URL, payload);
}

void bsp_wifi_init(void)
{
    s_wifi_event_group = xEventGroupCreate();

    bsp_nvs_flash_init();

    // 初始化事件循环，并注册回调函数
    ESP_ERROR_CHECK(esp_event_loop_create_default());
    ESP_ERROR_CHECK(esp_event_handler_register(WIFI_PROV_EVENT,
ESP_EVENT_ANY_ID, &event_handler, NULL));
```

```
    ESP_ERROR_CHECK(esp_event_handler_register(WIFI_EVENT,
ESP_EVENT_ANY_ID, &event_handler, NULL));
    ESP_ERROR_CHECK(esp_event_handler_register(IP_EVENT,
IP_EVENT_STA_GOT_IP, &event_handler, NULL));

    // 初始化 TCP/IP 栈
    ESP_ERROR_CHECK(esp_netif_init());
    esp_netif_t *wifi_netif = esp_netif_create_default_wifi_sta();
    assert(wifi_netif);

    // 设置用户名和密码，并启动 WiFi
    wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
    ESP_ERROR_CHECK(esp_wifi_init(&cfg));

    // 初始化配网 Manager
    wifi_prov_mgr_config_t config = {
        .scheme = wifi_prov_scheme_ble,
        .scheme_event_handler =
WIFI_PROV_SCHEME_BLE_EVENT_HANDLER_FREE_BTDM};

    ESP_ERROR_CHECK(wifi_prov_mgr_init(config));

    // 检查是否已经配网
    bool provisioned = false;
    ESP_ERROR_CHECK(wifi_prov_mgr_is_provisioned(&provisioned));
    if (!provisioned)
    {
        // 开始配网
        // 获取设备名称
        char service_name[12];
        get_device_service_name(service_name, sizeof(service_name));

        // 设置 BLE 加密方式
        wifi_prov_security_t security = WIFI_PROV_SECURITY_1;
        const char *pop = "abcd1234";

        // 设置 WiFi 的 UUID
        uint8_t custom_service_uuid[] = {
            0xb4, 0xdf, 0x5a, 0x1c,
            0x3f, 0x6b, 0xf4, 0xbf,
            0xea, 0x4a, 0x82, 0x03,
            0x04, 0x90, 0x1a, 0x02,
        };
        wifi_prov_scheme_ble_set_service_uuid(custom_service_uuid);
```

```
        wifi_prov_security1_params_t *sec_params = pop;
        const char *username  = NULL;
        const char *service_key = NULL;

        ESP_ERROR_CHECK(wifi_prov_mgr_start_provisioning(security,
(const void *) sec_params, service_name, service_key));
        wifi_prov_print_qr(service_name, username, pop,
PROV_TRANSPORT_BLE);
    }
    else
    {
        // 已经配网，直接连接 WiFi
        wifi_prov_mgr_deinit();
        ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_STA));
        ESP_ERROR_CHECK(esp_wifi_start());
    }

    xEventGroupWaitBits(s_wifi_event_group, WIFI_CONNECTED_BIT,
pdFALSE, pdTRUE, portMAX_DELAY);
}
```

3.2.5 BSP 总初始化

1）bsp.h

```
#ifndef BSP_H
#define BSP_H

#include "bsp_config.h"
#include "bsp_wifi.h"
#include "bsp_codec.h"
#include "bsp_led.h"
void bsp_init(void);

const char *bsp_get_mac_address(void);

const char *bsp_get_uuid(void);

/**
 * @brief 获取 OTA 中的 body
 *
 * @return char* 使用完成需要 free
 */
char *bsp_get_json(void);
```

```
#endif
```

2）bsp.c

```
#include "bsp.h"
#include <esp_mac.h>
#include <esp_random.h>
#include <cJSON.h>
#include <esp_system.h>
#include <esp_chip_info.h>
#include <esp_app_desc.h>
#include <esp_partition.h>
#include <esp_ota_ops.h>
#include <esp_psram.h>
#include <esp_flash.h>
#include "nvs_settings.h"


static char *uuid = NULL;
static char *mac_addr = NULL;

void bsp_init(void)
{
    bsp_led_init();
    bsp_led_set_state(LED_STATE_STARTING);

    bsp_codec_init();
    bsp_codec_set_mic_gain(10);
    bsp_codec_set_volume(60);
    bsp_wifi_init();
}

const char *bsp_get_mac_address(void)
{
    if (mac_addr)
    {
        return mac_addr;
    }

    mac_addr = malloc(18);
    assert(mac_addr);
    // 首先查询 mac 地址
    uint8_t mac[6];
    ESP_ERROR_CHECK(esp_read_mac(mac, ESP_MAC_WIFI_STA));
    snprintf(mac_addr, 18, "%02x:%02x:%02x:%02x:%02x:%02x", mac[0],
mac[1], mac[2], mac[3], mac[4], mac[5]);
```

```
    return mac_addr;
}

const char *bsp_get_uuid(void)
{
    if (uuid)
    {
        return uuid;
    }

    // 尝试读取 UUID
    settings_t *settings = nvs_settings_create();
    uuid = nvs_settings_get_string(settings, "uuid");
    if (uuid)
    {
        nvs_settings_free(settings);
        return uuid;
    }

    // 生成新的 UUID 并保存到 FLASH
    uuid = malloc(37);
    assert(uuid);
    uint8_t bytes[16];
    for (int i = 0; i < 16; i++)
    {
        bytes[i] = esp_random() & 0xFF;
    }
    // 设置版本和变体
    bytes[6] = (bytes[6] & 0x0F) | 0x40; // version 4
    bytes[8] = (bytes[8] & 0x3F) | 0x80; // variant 10

    snprintf(uuid, 37,
"%02x%02x%02x%02x-%02x%02x-%02x%02x-%02x%02x-%02x%02x%02x%02x%02x%02x",
             bytes[0], bytes[1], bytes[2], bytes[3],
             bytes[4], bytes[5], bytes[6], bytes[7],
             bytes[8], bytes[9], bytes[10], bytes[11],
             bytes[12], bytes[13], bytes[14], bytes[15]);

    // 保存到 FLASH
    nvs_settings_set_string(settings, "uuid", uuid);
    nvs_settings_free(settings);

    return uuid;
}
```

```
char *bsp_get_json(void)
{
    cJSON *root = cJSON_CreateObject();
    assert(root);
    cJSON_AddStringToObject(root, "uuid", bsp_get_uuid());
    cJSON_AddNumberToObject(root, "version", 2);

    // TODO)) 动态获取 PSRAM 大小
    cJSON_AddNumberToObject(root, "psram_size", esp_psram_get_size());
    cJSON_AddStringToObject(root, "chip_model_name", "esp32s3");
    cJSON_AddNumberToObject(root, "flash_size", 8 * 1024 * 1024);
    cJSON_AddStringToObject(root, "mac_address",
bsp_get_mac_address());
    cJSON_AddNumberToObject(root, "minimum_free_heap_size",
esp_get_minimum_free_heap_size());

    // application
    cJSON *application = cJSON_CreateObject();
    cJSON_AddItemToObject(root, "application", application);

    const esp_app_desc_t *app_desc = esp_app_get_description();
    cJSON_AddStringToObject(application, "compile_time",
app_desc->date);
    cJSON_AddStringToObject(application, "version", app_desc->version);
    cJSON_AddStringToObject(application, "name",
app_desc->project_name);
    cJSON_AddStringToObject(application, "idf_version",
app_desc->idf_ver);
    cJSON_AddStringToObject(application, "elf_sha256",
esp_app_get_elf_sha256_str());

    // board
    cJSON *board = cJSON_CreateObject();
    cJSON_AddItemToObject(root, "board", board);

    cJSON_AddNumberToObject(board, "chanel", BSP_CODEC_CHANNEL_COUNT);
    cJSON_AddStringToObject(board, "ip", "192.168.1.100");
    cJSON_AddStringToObject(board, "ssid", "atguigu");
    cJSON_AddNumberToObject(board, "rssi", -50);
    cJSON_AddStringToObject(board, "mac", bsp_get_mac_address());
    cJSON_AddStringToObject(board, "name", "bread-compact-wifi");
    cJSON_AddStringToObject(board, "type", "bread-compact-wifi");
```

```
    // chip_info
    cJSON *chip_info = cJSON_CreateObject();
    cJSON_AddItemToObject(root, "chip_info", chip_info);

    esp_chip_info_t chip;
    esp_chip_info(&chip);
    cJSON_AddNumberToObject(chip_info, "model", chip.model);
    cJSON_AddNumberToObject(chip_info, "features", chip.features);
    cJSON_AddNumberToObject(chip_info, "cores", chip.cores);
    cJSON_AddNumberToObject(chip_info, "revision", chip.revision);

    // partition_table
    cJSON *partition_table = cJSON_CreateArray();
    cJSON_AddItemToObject(root, "partition_table", partition_table);

    esp_partition_iterator_t it =
esp_partition_find(ESP_PARTITION_TYPE_ANY, ESP_PARTITION_SUBTYPE_ANY,
NULL);
    while (it)
    {
        const esp_partition_t *curr_par = esp_partition_get(it);
        // 处理这个 partition 信息
        cJSON *partition = cJSON_CreateObject();
        cJSON_AddItemToArray(partition_table, partition);

        cJSON_AddNumberToObject(partition, "address",
curr_par->address);
        cJSON_AddNumberToObject(partition, "size", curr_par->size);
        cJSON_AddNumberToObject(partition, "type", curr_par->type);
        cJSON_AddNumberToObject(partition, "subtype",
curr_par->subtype);
        cJSON_AddStringToObject(partition, "label", curr_par->label);

        // 迭代
        it = esp_partition_next(it);
    }
    esp_partition_iterator_release(it);

    // ota 模块
    const esp_partition_t *running_partition =
esp_ota_get_running_partition();
    cJSON *ota = cJSON_CreateObject();
    cJSON_AddItemToObject(root, "ota", ota);
    cJSON_AddStringToObject(ota, "label", running_partition->label);
```

```
    char *json_str = cJSON_PrintUnformatted(root);
    cJSON_Delete(root);
    return json_str;
}
```

3.3 音频接口

音频处理模块需要实现从 mic 输入音频数据，进行唤醒词识别和人声识别，将识别到的人声进行 opus 编码；也需要将服务器发来的 opus 数据进行解码，并在扬声器播放，数据链路如下：

```
MIC 输入 -> 唤醒识别+人声识别 -> opus 编码 -> 读取接口
写入接口 -> opus 解码 -> 扬声器
```

各个模块之间使用环形缓存连接。语音识别和人声识别的过程，我们使用 esp 官方提供的语音识别模块。我们不需要语音指令，该模块仅仅只要 960k Flash 空间即可实现。

3.3.1 语音识别

1）audio_sr.h

```
#ifndef AUDIO_SR_H
#define AUDIO_SR_H

#include "esp_afe_sr_iface.h"
#include <freertos/FreeRTOS.h>
#include <freertos/ringbuf.h>

typedef struct audio_sr audio_sr_t;

/**
 * @brief 创建一个语音识别实例
 *
 * @return audio_sr_t* 语音识别实例
 */
audio_sr_t *audio_sr_create(void);

/**
 * @brief 开启语音识别后台任务
 *
 * @param audio_sr 语音识别实例
 */
void audio_sr_start(audio_sr_t *audio_sr);
```

```
/**
 * @brief 设置唤醒词触发回调
 *
 * @param audio_sr 语音识别实例
 * @param callback 唤醒词出发以后调用的回调函数
 * @param arg 回调函数参数
 */
void audio_sr_set_wakenet_callback(audio_sr_t *audio_sr, void
(*callback)(void *), void *arg);

/**
 * @brief 设置人声检测回调
 *
 * @param audio_sr 语音识别实例
 * @param callback 人声检测变化后调用的回调函数
 * @param arg 参数
 */
void audio_sr_set_vad_callback(audio_sr_t *audio_sr, void
(*callback)(void *, vad_state_t vad_state), void *arg);

/**
 * @brief 设置采集到的人声的环形缓存
 *
 * @param audio_sr 语音识别实例
 * @param buf 环形缓存
 */
void audio_sr_set_output_buffer(audio_sr_t *audio_sr, RingbufHandle_t
buf);

/**
 * @brief 重置唤醒状态
 *
 * @param audio_sr 语音识别实例
 */
void audio_sr_reset_wakenet(audio_sr_t *audio_sr);

#endif
```

2）audio_sr.c

```
#include "audio_sr.h"
#include "esp_afe_sr_models.h"
#include <esp_err.h>
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>
#include "bsp_codec.h"
```

```
// #include "object.h"
#include <string.h>
#include <stdlib.h>
#include <esp_log.h>

#define TAG "[AUDIO_PROCESSOR] SR"

struct audio_sr
{
    esp_afe_sr_iface_t *afe_handle;
    esp_afe_sr_data_t *afe_data;
    bool is_waken;
    vad_state_t last_state;

    void (*wakenet_callback)(void *);
    void *wakenet_callback_arg;

    void (*vad_callback)(void *, vad_state_t);
    void *vad_callback_arg;

    RingbufHandle_t output_buffer;
};

static void audio_sr_feed_task(void *arg)
{
    ESP_LOGI(TAG, "Feed task created");
    audio_sr_t *audio_sr = (audio_sr_t *)arg;
    esp_afe_sr_iface_t *afe_handle = audio_sr->afe_handle;
    esp_afe_sr_data_t *afe_data = audio_sr->afe_data;

    int feed_chunksize = afe_handle->get_feed_chunksize(afe_data);
    int feed_nch = afe_handle->get_feed_channel_num(afe_data);
    size_t feed_chunksize_bytes = feed_chunksize * feed_nch *
sizeof(int16_t);
    int16_t *feed_buff = (int16_t *)malloc(feed_chunksize_bytes);

    // 用数据填满缓冲区
    while (1)
    {
        bsp_codec_read(feed_buff, feed_chunksize_bytes);
        afe_handle->feed(afe_data, feed_buff);
    }

    vTaskDelete(NULL);
```

```
}

static void audio_sr_fetch_task(void *arg)
{
    ESP_LOGI(TAG, "Fetch task created");
    audio_sr_t *audio_sr = (audio_sr_t *)arg;
    esp_afe_sr_iface_t *afe_handle = audio_sr->afe_handle;
    esp_afe_sr_data_t *afe_data = audio_sr->afe_data;

    int fetch_chunksize = afe_handle->get_fetch_chunksize(afe_data);
    int fetch_nch = afe_handle->get_fetch_channel_num(afe_data);
    size_t fetch_chunksize_bytes = fetch_chunksize * fetch_nch *
sizeof(int16_t);

    while (1)
    {
        afe_fetch_result_t *result = afe_handle->fetch(afe_data);
        int16_t *raw_data = result->raw_data;
        vad_state_t vad_state = result->vad_state;
        wakenet_state_t wakeup_state = result->wakeup_state;

        if (wakeup_state == WAKENET_DETECTED)
        {
            // 检测到唤醒词
            audio_sr->is_waken = true;
            if (audio_sr->wakenet_callback)
            {
                audio_sr->wakenet_callback(audio_sr->wakenet_callback_a
rg);
            }
        }

        if (audio_sr->is_waken)
        {
            if (vad_state != audio_sr->last_state)
            {
                audio_sr->last_state = vad_state;
                if (audio_sr->vad_callback)
                {
                    audio_sr->vad_callback(audio_sr->vad_callback_arg,
audio_sr->last_state);
                }
                if (vad_state == VAD_SPEECH && result->vad_cache_size >
0)
```

```
                {
                    // 将 vad cache 写入缓存
                    xRingbufferSend(audio_sr->output_buffer,
result->vad_cache, result->vad_cache_size, 0);
                }
            }
        }
        else
        {
            if (audio_sr->last_state == VAD_SPEECH)
            {
                audio_sr->last_state = VAD_SILENCE;
                if (audio_sr->vad_callback)
                {
                    audio_sr->vad_callback(audio_sr->vad_callback_arg,
audio_sr->last_state);
                }
            }
        }

        if (audio_sr->last_state == VAD_SPEECH)
        {
            // 将人声放到环形缓存
            xRingbufferSend(audio_sr->output_buffer, raw_data,
fetch_chunksize_bytes, 0);
        }
    }

    vTaskDelete(NULL);
}

audio_sr_t *audio_sr_create(void)
{
    audio_sr_t *audio_sr = malloc(sizeof(audio_sr_t));
    assert(audio_sr);
    memset(audio_sr, 0, sizeof(audio_sr_t));

    srmodel_list_t *models = esp_srmodel_init("model");
    afe_config_t *afe_config = afe_config_init("M", models,
AFE_TYPE_SR, AFE_MODE_LOW_COST);

    // 自定义 afe 配置
    afe_config->aec_init = false;
    afe_config->se_init = false;
```

```
    afe_config->ns_init = false;
    afe_config->vad_mode = VAD_MODE_3;
    // afe_config->vad_min_speech_ms = 64;
    // afe_config->vad_delay_ms = 64;
    afe_config->vad_min_noise_ms = 500;
    afe_config->wakenet_mode = DET_MODE_90;
    afe_config->memory_alloc_mode = AFE_MEMORY_ALLOC_MORE_PSRAM;

    // 获取句柄
    audio_sr->afe_handle = esp_afe_handle_from_config(afe_config);
    assert(audio_sr->afe_handle);
    // 创建实例
    audio_sr->afe_data =
audio_sr->afe_handle->create_from_config(afe_config);
    assert(audio_sr->afe_data);

    return audio_sr;
}

void audio_sr_start(audio_sr_t *audio_sr)
{
    xTaskCreate(audio_sr_fetch_task, "fetch_task", 4096, audio_sr, 6,
NULL);
    xTaskCreate(audio_sr_feed_task, "feek_task", 4096, audio_sr, 5,
NULL);
}

void audio_sr_set_wakenet_callback(audio_sr_t *audio_sr, void
(*callback)(void *), void *arg)
{
    audio_sr->wakenet_callback = callback;
    audio_sr->wakenet_callback_arg = arg;
}

void audio_sr_set_vad_callback(audio_sr_t *audio_sr, void
(*callback)(void *, vad_state_t vad_state), void *arg)
{
    audio_sr->vad_callback = callback;
    audio_sr->vad_callback_arg = arg;
}

void audio_sr_set_output_buffer(audio_sr_t *audio_sr, RingbufHandle_t
buf)
{
```

```
    audio_sr->output_buffer = buf;
}

void audio_sr_reset_wakenet(audio_sr_t *audio_sr)
{
    audio_sr->is_waken = false;
}
```

3.3.2 Opus 编码器

1）audio_encoder.h

```
#ifndef AUDIO_ENCODER_H
#define AUDIO_ENCODER_H

#include <freertos/FreeRTOS.h>
#include <freertos/ringbuf.h>

typedef struct audio_encoder audio_encoder_t;

/**
 * @brief 创建编码器实例
 *
 * @return audio_encoder_t* 编码器实例指针
 */
audio_encoder_t *audio_encoder_create(void);

/**
 * @brief 开启编码任务
 *
 * @param audio_encoder 编码器实例
 */
void audio_encoder_start(audio_encoder_t *audio_encoder);

/**
 * @brief 设置编码器的输入 ringbuf
 *
 * @param audio_encoder 编码器实例
 * @param buf 输入的 ringbuf
 */
void audio_encoder_set_input_buffer(audio_encoder_t *audio_encoder,
RingbufHandle_t buf);

/**
 * @brief 设置编码器的输出 ringbuf
```

```
 *
 * @param audio_encoder 编码器实例
 * @param buf 输出的 ringbuf
 */
void audio_encoder_set_output_buffer(audio_encoder_t *audio_encoder,
RingbufHandle_t buf);

#endif
```

2）audio_encoder.c

```
#include "audio_encoder.h"
#include "esp_audio_enc.h"
#include "esp_audio_enc_default.h"
#include "esp_audio_enc_reg.h"
#include <stdlib.h>
#include <string.h>
#include <esp_err.h>
#include <esp_log.h>
#include "bsp_config.h"
#include <esp_heap_caps.h>

#define TAG "[AUDIO_PROCESSOR] Encoder"

struct audio_encoder
{
    esp_audio_enc_handle_t encoder;
    bool is_running;

    RingbufHandle_t input_buf;
    RingbufHandle_t output_buf;
};

static void copy_from_ringbuf(uint8_t *buf, int size, RingbufHandle_t
ringbuf)
{
    uint8_t *read_buf = NULL;
    size_t read_size = 0;
    while (size > 0)
    {
        read_buf = xRingbufferReceiveUpTo(ringbuf, &read_size,
portMAX_DELAY, size);
        assert(read_buf);

        memcpy(buf, read_buf, read_size);
        vRingbufferReturnItem(ringbuf, read_buf);
```

```
        buf += read_size;
        size -= read_size;
    }
}

static void audio_encoder_task(void *arg)
{
    ESP_LOGI(TAG, "Encoder task created");
    audio_encoder_t *audio_encoder = (audio_encoder_t *)arg;
    esp_audio_enc_handle_t encoder = audio_encoder->encoder;

    // 从环形缓存中读取数据，编码并写出
    int read_size = 0;
    int write_size = 0;
    ESP_ERROR_CHECK(esp_audio_enc_get_frame_size(encoder, &read_size,
&write_size));

    uint8_t *input_buf = (uint8_t *)heap_caps_malloc(read_size,
MALLOC_CAP_SPIRAM);
    assert(input_buf);

    uint8_t *output_buf = (uint8_t *)heap_caps_malloc(write_size,
MALLOC_CAP_SPIRAM);
    assert(output_buf);

    esp_audio_enc_in_frame_t in_frame = {
        .buffer = input_buf,
        .len = read_size,
    };

    esp_audio_enc_out_frame_t out_frame = {
        .buffer = output_buf,
        .len = write_size,
    };

    while (audio_encoder->is_running)
    {
        // 从输入缓存中读取数据
        copy_from_ringbuf(input_buf, read_size,
audio_encoder->input_buf);

        // 编码数据
        ESP_ERROR_CHECK(esp_audio_enc_process(encoder, &in_frame,
&out_frame));
```

```
        // 写出编码后的数据
        assert(xRingbufferSend(audio_encoder->output_buf, output_buf,
out_frame.encoded_bytes, portMAX_DELAY) == pdTRUE);
    }
    free(input_buf);
    free(output_buf);
    vTaskDelete(NULL);
}

audio_encoder_t *audio_encoder_create(void)
{
    audio_encoder_t *audio_encoder = (audio_encoder_t
*)malloc(sizeof(audio_encoder_t));
    assert(audio_encoder);

    // 注册默认编码器
    ESP_ERROR_CHECK(esp_opus_enc_register());

    // 生成配置文件
    esp_opus_enc_config_t opus_config = {
        .application_mode = ESP_OPUS_ENC_APPLICATION_AUDIO,
        .bitrate = 32000,
        .bits_per_sample = BSP_CODEC_BIT_PER_SAMPLE,
        .channel = BSP_CODEC_CHANNEL_COUNT,
        .complexity = 6,
        .enable_dtx = false,
        .enable_fec = true,
        .enable_vbr = true,
        .frame_duration = ESP_OPUS_ENC_FRAME_DURATION_60_MS,
        .sample_rate = BSP_CODEC_SAMPLE_RATE,
    };

    esp_audio_enc_config_t enc_config = {
        .type = ESP_AUDIO_TYPE_OPUS,
        .cfg = &opus_config,
        .cfg_sz = sizeof(esp_opus_enc_config_t),
    };
    ESP_ERROR_CHECK(esp_audio_enc_open(&enc_config,
&audio_encoder->encoder));

    // 创建编码器

    return audio_encoder;
```

```
}

void audio_encoder_start(audio_encoder_t *audio_encoder)
{
    audio_encoder->is_running = true;
    assert(xTaskCreatePinnedToCoreWithCaps(audio_encoder_task,
"enc_task", 1024 * 32, audio_encoder, 5, NULL, 1, MALLOC_CAP_SPIRAM) ==
pdTRUE);
}

void audio_encoder_set_input_buffer(audio_encoder_t *audio_encoder,
RingbufHandle_t buf)
{
    assert(audio_encoder);
    assert(buf);
    audio_encoder->input_buf = buf;
}

void audio_encoder_set_output_buffer(audio_encoder_t *audio_encoder,
RingbufHandle_t buf)
{
    assert(audio_encoder);
    assert(buf);
    audio_encoder->output_buf = buf;
}
```

3.3.3 Opus 解码器

1）audio_decoder.h

```
#ifndef AUDIO_DECODER_H
#define AUDIO_DECODER_H

#include <freertos/FreeRTOS.h>
#include <freertos/ringbuf.h>

typedef struct audio_decoder audio_decoder_t;

audio_decoder_t *audio_decoder_create(void);

void audio_decoder_start(audio_decoder_t *audio_decoder);

void audio_decoder_set_input_buffer(audio_decoder_t *audio_decoder,
RingbufHandle_t buf);
```

```
void audio_decoder_set_output_buffer(audio_decoder_t *audio_decoder,
RingbufHandle_t buf);


#endif
```

2）audio_decoder.c

```
#include "audio_decoder.h"
#include "esp_audio_dec.h"
#include "esp_audio_dec_default.h"
#include "esp_audio_dec_reg.h"
#include <esp_heap_caps.h>
#include <stdlib.h>
#include <string.h>
#include <esp_err.h>
#include <bsp_config.h>
#include <esp_log.h>

#define TAG "[AUDIO_PROCESSOR] Deocder"

struct audio_decoder
{
    esp_audio_dec_handle_t decoder;
    bool is_running;

    RingbufHandle_t input_buf;
    RingbufHandle_t output_buf;
};

static void audio_decoder_task(void *arg)
{
    ESP_LOGI(TAG, "Decoder task created");
    audio_decoder_t *audio_decoder = (audio_decoder_t *)arg;
    uint8_t *read_buf = NULL;
    size_t read_size = 0;

    esp_audio_dec_in_raw_t in_frame = {0};

    uint32_t out_len = BSP_CODEC_BIT_PER_SAMPLE *
BSP_CODEC_CHANNEL_COUNT * BSP_CODEC_SAMPLE_RATE / 8 / 1000 * 60;
    esp_audio_dec_out_frame_t out_frame = {
        .buffer = heap_caps_malloc(out_len, MALLOC_CAP_SPIRAM),
        .len = out_len,
    };
```

```
    while (audio_decoder->is_running)
    {
        // 读取环形缓存
        read_buf = xRingbufferReceive(audio_decoder->input_buf,
&read_size, portMAX_DELAY);
        assert(read_buf);
        in_frame.buffer = read_buf;
        in_frame.len = read_size;
        while (in_frame.len > 0)
        {
            // 解码
            int ret = esp_audio_dec_process(audio_decoder->decoder,
&in_frame, &out_frame);
            if (ret == ESP_AUDIO_ERR_BUFF_NOT_ENOUGH)
            {
                ESP_LOGE(TAG, "Decoder buffer not enough; needed %lu,
current %lu", out_frame.needed_size, out_frame.len);
                abort();
            }
            ESP_ERROR_CHECK(ret);

            // 写到输出缓存
            ESP_LOGD(TAG, "Decoded %lu bytes", out_frame.decoded_size);
            assert(xRingbufferSend(audio_decoder->output_buf,
out_frame.buffer, out_frame.decoded_size, portMAX_DELAY) == pdTRUE);

            in_frame.buffer += in_frame.consumed;
            in_frame.len -= in_frame.consumed;
        }

        vRingbufferReturnItem(audio_decoder->input_buf, read_buf);
    }
}

audio_decoder_t *audio_decoder_create(void)
{
    audio_decoder_t *audio_decoder = (audio_decoder_t
*)malloc(sizeof(audio_decoder_t));
    assert(audio_decoder);
    memset(audio_decoder, 0, sizeof(audio_decoder_t));
    // docoder 初始化

    // 注册解码器
    ESP_ERROR_CHECK(esp_opus_dec_register());
```

```
    esp_opus_dec_cfg_t opus_dec_cfg = {
        .channel = BSP_CODEC_CHANNEL_COUNT,
        .sample_rate = BSP_CODEC_SAMPLE_RATE,
        // .frame_duration = ESP_OPUS_DEC_FRAME_DURATION_60_MS,
        .self_delimited = false,
    };

    esp_audio_dec_cfg_t dec_cfg = {
        .cfg = &opus_dec_cfg,
        .cfg_sz = sizeof(esp_opus_dec_cfg_t),
        .type = ESP_AUDIO_TYPE_OPUS,
    };

    ESP_ERROR_CHECK(esp_audio_dec_open(&dec_cfg,
&audio_decoder->decoder));

    return audio_decoder;
}

void audio_decoder_start(audio_decoder_t *audio_decoder)
{
    assert(audio_decoder);
    audio_decoder->is_running = true;
    assert(xTaskCreatePinnedToCoreWithCaps(audio_decoder_task,
"dec_task", 1024 * 32, audio_decoder, 5, NULL, 1, MALLOC_CAP_SPIRAM) ==
pdTRUE);
}

void audio_decoder_set_input_buffer(audio_decoder_t *audio_decoder,
RingbufHandle_t buf)
{
    assert(audio_decoder);
    assert(buf);
    audio_decoder->input_buf = buf;
}

void audio_decoder_set_output_buffer(audio_decoder_t *audio_decoder,
RingbufHandle_t buf)
{
    assert(audio_decoder);
    assert(buf);
    audio_decoder->output_buf = buf;
}
```

3.3.4 音频处理模块

1）audio_processor.h

```
#ifndef AUDIO_PROCESSOR_H
#define AUDIO_PROCESSOR_H

#include <esp_types.h>
#include "esp_afe_sr_models.h"

void audio_processor_init(void);

void audio_processor_start(void);

/**
 * @brief 读取经过处理的音频数据
 *
 * @param size_p 返回读取长度
 * @return void* 返回的缓存指针，使用完成后需要 free
 */
void *audio_processor_read(size_t *size_p);

void audio_processor_write(void *buf, size_t size);

void audio_processor_set_wakenet_callback(void (*callback)(void *),
void *arg);

void audio_processor_set_vad_callback(void (*callback)(void *,
vad_state_t vad_state), void *arg);

void audio_processor_reset_wakenet();

#endif
```

2）audio_processor.c

```
#include "audio_processor.h"
#include "audio_sr.h"
#include "audio_decoder.h"
#include "audio_encoder.h"
#include "bsp_codec.h"
#include <stdlib.h>
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>
#include <freertos/ringbuf.h>
```

```
#include <esp_heap_caps.h>
#include <string.h>
#include <esp_log.h>

#define TAG "audio_processor"

typedef struct
{
    audio_encoder_t *audio_encoder;
    audio_decoder_t *audio_decoder;
    audio_sr_t *audio_sr;

    RingbufHandle_t mic_buf;
    RingbufHandle_t enc_buf;
    RingbufHandle_t dec_buf;
    RingbufHandle_t speaker_buf;

} audio_processor_t;

static audio_processor_t *s_audio_processor = NULL;

static void audio_processor_speak_task(void *arg)
{
    void *buf = NULL;
    size_t len = 0;
    while (1)
    {
        buf = xRingbufferReceive(s_audio_processor->speaker_buf, &len,
portMAX_DELAY);
        if (buf != NULL)
        {
            ESP_LOGD(TAG, "speak task, len %u", len);
            bsp_codec_write(buf, len);
            vRingbufferReturnItem(s_audio_processor->speaker_buf, buf);
        }
    }
}

void audio_processor_init(void)
{
    if (s_audio_processor)
    {
        return;
    }
```

```
    s_audio_processor = (audio_processor_t
*)malloc(sizeof(audio_processor_t));
    assert(s_audio_processor);

    s_audio_processor->mic_buf = xRingbufferCreateWithCaps(1024 * 8,
RINGBUF_TYPE_BYTEBUF, MALLOC_CAP_SPIRAM);
    assert(s_audio_processor->mic_buf);
    s_audio_processor->speaker_buf = xRingbufferCreateWithCaps(1024 *
8, RINGBUF_TYPE_NOSPLIT, MALLOC_CAP_SPIRAM);
    assert(s_audio_processor->speaker_buf);
    s_audio_processor->dec_buf = xRingbufferCreateWithCaps(1024 * 8,
RINGBUF_TYPE_NOSPLIT, MALLOC_CAP_SPIRAM);
    assert(s_audio_processor->dec_buf);
    s_audio_processor->enc_buf = xRingbufferCreateWithCaps(1024 * 8,
RINGBUF_TYPE_NOSPLIT, MALLOC_CAP_SPIRAM);
    assert(s_audio_processor->enc_buf);

    s_audio_processor->audio_sr = audio_sr_create();
    audio_sr_set_output_buffer(s_audio_processor->audio_sr,
s_audio_processor->mic_buf);

    s_audio_processor->audio_encoder = audio_encoder_create();
    audio_encoder_set_input_buffer(s_audio_processor->audio_encoder,
s_audio_processor->mic_buf);
    audio_encoder_set_output_buffer(s_audio_processor->audio_encoder,
s_audio_processor->enc_buf);

    s_audio_processor->audio_decoder = audio_decoder_create();
    audio_decoder_set_input_buffer(s_audio_processor->audio_decoder,
s_audio_processor->dec_buf);
    audio_decoder_set_output_buffer(s_audio_processor->audio_decoder,
s_audio_processor->speaker_buf);
}

void audio_processor_start(void)
{
    assert(xTaskCreatePinnedToCoreWithCaps(audio_processor_speak_task,
"spk_task", 1024 * 4, NULL, 5, NULL, 1, MALLOC_CAP_SPIRAM) == pdTRUE);
    audio_decoder_start(s_audio_processor->audio_decoder);
    audio_encoder_start(s_audio_processor->audio_encoder);
    audio_sr_start(s_audio_processor->audio_sr);
}
```

```
void *audio_processor_read(size_t *size_p)
{
    size_t rean_len = 0;
    void *read_buf = xRingbufferReceive(s_audio_processor->enc_buf,
&rean_len, portMAX_DELAY);
    assert(read_buf);

    void *ret_buf = malloc(rean_len);
    assert(ret_buf);
    memcpy(ret_buf, read_buf, rean_len);
    *size_p = rean_len;
    vRingbufferReturnItem(s_audio_processor->enc_buf, read_buf);

    return ret_buf;
}

void audio_processor_write(void *buf, size_t size)
{
    assert(xRingbufferSend(s_audio_processor->dec_buf, buf, size,
portMAX_DELAY) == pdTRUE);
}

void audio_processor_set_wakenet_callback(void (*callback)(void *),
void *arg)
{
    assert(s_audio_processor);
    assert(s_audio_processor->audio_sr);
    audio_sr_set_wakenet_callback(s_audio_processor->audio_sr,
callback, arg);
}

void audio_processor_set_vad_callback(void (*callback)(void *,
vad_state_t vad_state), void *arg)
{
    assert(s_audio_processor);
    assert(s_audio_processor->audio_sr);
    audio_sr_set_vad_callback(s_audio_processor->audio_sr, callback,
arg);
}

void audio_processor_reset_wakenet()
{
    assert(s_audio_processor);
    assert(s_audio_processor->audio_sr);
```

```
    audio_sr_reset_wakenet(s_audio_processor->audio_sr);
}
```

3.4 协议接口

协议层需要实现上面说的各种协议。例如向服务器发送连接信息，发送唤醒词

3.4.1 协议总接口

1）protocol.h

```
#ifndef PROTOCOL_H
#define PROTOCOL_H

#include <esp_types.h>
#include <cJSON.h>
#include "protocol_transport.h"

typedef enum
{
    PROTOCOL_ABORT_REASON_NONE = 0,
    PROTOCOL_ABORT_REASON_WAKEUP,
} protocol_abort_reason_t;

/**
 * @brief 文本回调函数
 *
 * @param text 文本
 */
typedef void (*text_callback_t)(char *text);

/**
 * @brief 动作回调函数
 */
typedef void (*action_callback_t)(void);

/**
 * @brief 音频回调函数
 *
 * @param data 音频数据
 * @param len 音频数据长度
 */
typedef void (*audio_callback_t)(void *data, size_t len);

typedef struct protocol protocol_t;
```

```
protocol_t *protocol_create(protocol_transport_type_t type);

/**
 * @brief 打开音频通道
 *
 * @param protocol 协议
 * @return true 成功
 * @return false 失败
 */
bool protocol_open_audio_channel(protocol_t *protocol);

/**
 * @brief 关闭音频通道
 *
 * @param protocol 协议
 */
void protocol_close_audio_channel(protocol_t *protocol);

bool protocol_is_audio_channel_open(protocol_t *protocol);

/**
 * @brief 发送音频数据
 *
 * @param data 音频数据
 * @param len 数据长度
 */
void protocol_send_audio(protocol_t *protocol, void *data, size_t len);
/**
 * @brief 建立信道
 *
 */
void protocol_send_hello(protocol_t *protocol);

/**
 * @brief 发送唤醒词
 *
 */
void protocol_send_wake_word(protocol_t *protocol, char *word);

/**
 * @brief 启动和停止监听
 *
 */
void protocol_send_start_listening(protocol_t *protocol);
```

```
void protocol_send_stop_listening(protocol_t *protocol);

/**
 * @brief 中断播放
 *
 * @param reason 中断原因
 */
void protocol_abort_speaking(protocol_t *protocol,
protocol_abort_reason_t reason);

/**
 * @brief 设置识别回调
 *
 * @param callback 回调函数
 */
void protocol_set_stt_callback(protocol_t *protocol, text_callback_t
callback);

/**
 * @brief 设置意图识别回调
 *
 * @param callback
 */
void protocol_set_llm_callback(protocol_t *protocol, text_callback_t
callback);

/**
 * @brief 播放开始回调
 *
 * @param callback 回调函数
 */
void protocol_set_tts_start_callback(protocol_t *protocol,
action_callback_t callback);

/**
 * @brief 播放结束回调
 *
 * @param callback 回调函数
 */
void protocol_set_tts_stop_callback(protocol_t *protocol,
action_callback_t callback);

/**
 * @brief 播放每句话的回调
```

```
 *
 * @param callback 回调函数
 */
void protocol_set_tts_sentence_start_callback(protocol_t *protocol,
text_callback_t callback);

void protocol_set_iot_callback(protocol_t *protocol, text_callback_t
callback);
/**
 * @brief 音频回调
 *
 * @param callback 回调函数
 */
void protocol_set_audio_in_callback(protocol_t *protocol,
audio_callback_t callback);

/**
 * @brief 音频通道关闭回调
 *
 * @param callback 回调函数
 */
void protocol_set_audio_channel_close_callback(protocol_t *protocol,
action_callback_t callback);

void protocol_send_iot_descriptor(protocol_t *protocol, cJSON
*discriptor_json);

void protocol_send_iot_state(protocol_t *protocol, cJSON *state_json);

#endif
```

2）protocol.c

```
#include "protocol.h"
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <esp_log.h>

#include "bsp_config.h"
#include <freertos/FreeRTOS.h>
#include <freertos/event_groups.h>

#define TAG "Protocol"
#define PROTOCOL_EVENT_HELLO (1 << 0)
```

```
struct protocol
{
    protocol_transport_t *transport;
    text_callback_t stt_callback;
    text_callback_t llm_callback;
    action_callback_t tts_start_callback;
    text_callback_t tts_sentence_start_callback;
    action_callback_t tts_stop_callback;
    action_callback_t audio_channel_close_callback;
    audio_callback_t audio_in_callback;
    text_callback_t iot_callback;

    char *session_id;
    int server_sample_rate;

    EventGroupHandle_t event_group;
};

static void protocol_audio_in_callback(void *arg, void *data, size_t
len);
static void protocol_channel_close_callback(void *arg);
static void protocol_text_callback(void *arg, char *text);

static protocol_transport_callbacks_t callbacks = {
    .channel_close_callback = protocol_channel_close_callback,
    .audio_callback = protocol_audio_in_callback,
    .text_callback = protocol_text_callback,
};

// protocol_audio_in_callback、protocol_text_callback、
protocol_channel_close_callback
// 三个函数是供协议传输层调用的回调
// 从协议传输层上来的数据，一般两种类型，音频和文本。还有一种可能是服务器关闭
了信道连接
static void protocol_audio_in_callback(void *arg, void *data, size_t
len)
{
    protocol_t *protocol = arg;
    if (protocol->audio_in_callback)
    {
        protocol->audio_in_callback(data, len);
    }
}
```

```
static void protocol_text_callback(void *arg, char *text)
{
    protocol_t *protocol = arg;
    cJSON *root = cJSON_Parse(text);
    if (!root)
    {
        ESP_LOGE(TAG, "Failed to parse JSON");
        ESP_LOGE(TAG, "JSON: %s", text);
        return;
    }

    cJSON *type_obj = cJSON_GetObjectItem(root, "type");
    if (!cJSON_IsString(type_obj))
    {
        ESP_LOGE(TAG, "Failed to parse reply type");
        cJSON_Delete(root);
        return;
    }

    if (strcmp(type_obj->valuestring, "hello") == 0)
    {
        // 解析 hello 数据
        cJSON *audio_params_obj = cJSON_GetObjectItem(root,
"audio_params");
        cJSON *session_id_obj = cJSON_GetObjectItem(root,
"session_id");

        if (!audio_params_obj || !cJSON_IsString(session_id_obj))
        {
            ESP_LOGE(TAG, "Invalid hello message, %s", text);
            cJSON_Delete(root);
            return;
        }
        protocol->session_id = strdup(session_id_obj->valuestring);

        cJSON *sample_rate_obj = cJSON_GetObjectItem(audio_params_obj,
"sample_rate");
        if (!cJSON_IsNumber(sample_rate_obj))
        {
            ESP_LOGE(TAG, "Invalid hello message, %s", text);
            cJSON_Delete(root);
            return;
        }
```

```
        protocol->server_sample_rate = sample_rate_obj->valueint;
        xEventGroupSetBits(protocol->event_group,
PROTOCOL_EVENT_HELLO);
    }
    else if (strcmp(type_obj->valuestring, "stt") == 0 &&
protocol->stt_callback)
    {
        // 解析 stt 数据
        cJSON *text_obj = cJSON_GetObjectItem(root, "text");
        if (!cJSON_IsString(text_obj))
        {
            ESP_LOGE(TAG, "Invalid stt message, %s", text);
            cJSON_Delete(root);
            return;
        }
        protocol->stt_callback(text_obj->valuestring);
    }
    else if (strcmp(type_obj->valuestring, "llm") == 0 &&
protocol->llm_callback)
    {
        // 解析 llm 数据
        cJSON *text_obj = cJSON_GetObjectItem(root, "text");
        if (!cJSON_IsString(text_obj))
        {
            ESP_LOGE(TAG, "Invalid stt message, %s", text);
            cJSON_Delete(root);
            return;
        }
        protocol->llm_callback(text_obj->valuestring);
    }
    else if (strcmp(type_obj->valuestring, "tts") == 0)
    {
        // 查看 state
        cJSON *state_obj = cJSON_GetObjectItem(root, "state");
        if (!cJSON_IsString(state_obj))
        {
            ESP_LOGE(TAG, "TTS state is not a string");
            cJSON_Delete(root);
            return;
        }

        if (strcmp(state_obj->valuestring, "start") == 0 &&
protocol->tts_start_callback)
        {
```

```
            protocol->tts_start_callback();
        }
        else if (strcmp(state_obj->valuestring, "stop") == 0 &&
protocol->tts_stop_callback)
        {
            protocol->tts_stop_callback();
        }
        else if (strcmp(state_obj->valuestring, "sentence_start") == 0
&& protocol->tts_sentence_start_callback)
        {
            cJSON *text_obj = cJSON_GetObjectItem(root, "text");
            if (!cJSON_IsString(text_obj))
            {
                ESP_LOGE(TAG, "Invalid stt message, %s", text);
                cJSON_Delete(root);
                return;
            }
            protocol->tts_sentence_start_callback(text_obj->valuestring
);
        }
    }
    else if (strcmp(type_obj->valuestring, "iot") == 0)
    {
        cJSON *command_array = cJSON_GetObjectItem(root, "commands");
        if (command_array)
        {
            protocol->iot_callback((char *)command_array);
        }
    }
    cJSON_Delete(root);
}

static void protocol_channel_close_callback(void *arg)
{
    protocol_t *protocol = arg;
    if (protocol->audio_channel_close_callback)
    {
        protocol->audio_channel_close_callback();
    }
}



// 检查能否发送文本
```

```
static bool protocol_can_send_text(protocol_t *protocol)
{
    if (!protocol->transport->is_audio_channel_open
|| !protocol->transport->send_text)
    {
        ESP_LOGE(TAG, "Send text not supported");
        return false;
    }

    if
(!protocol->transport->is_audio_channel_open(protocol->transport))
    {
        ESP_LOGE(TAG, "Audio channel not open");
        return false;
    }
    return true;
}

// 协议初始化
protocol_t *protocol_create(protocol_transport_type_t type)
{
    protocol_t *protocol = malloc(sizeof(protocol_t));
    assert(protocol);
    memset(protocol, 0, sizeof(protocol_t));

    protocol->event_group = xEventGroupCreate();
    assert(protocol->event_group);

    switch (type)
    {
    case PROTOCOL_TRANSPORT_TYPE_WEBSOCKET:
        protocol->transport =
protocol_transport_create_websocket(&callbacks, protocol);
        break;

    default:
        ESP_LOGE(TAG, "Unsupported protocol transport type");
        break;
    }

    if (protocol->transport->start)
    {
        protocol->transport->start(protocol->transport);
    }
```

```
    return protocol;
}



// protocol_open_audio_channel、protocol_close_audio_channel、
protocol_is_audio_channel_open
// 负责开启、关闭、检查信道状态
bool protocol_open_audio_channel(protocol_t *protocol)
{
    if (!protocol->transport->open_audio_channel)
    {
        ESP_LOGE(TAG, "Open audio channel not supported");
        return false;
    }

    return
protocol->transport->open_audio_channel(protocol->transport);
}

void protocol_close_audio_channel(protocol_t *protocol)
{
    if (!protocol->transport->close_audio_channel)
    {
        ESP_LOGE(TAG, "Close audio channel not supported");
        return;
    }
    protocol->transport->close_audio_channel(protocol->transport);
}

bool protocol_is_audio_channel_open(protocol_t *protocol)
{
    if (protocol->transport->is_audio_channel_open)
    {
        return
protocol->transport->is_audio_channel_open(protocol->transport);
    }

    return false;
}


// 一系列 send 函数，是单片机主动向服务器发送的请求
void protocol_send_audio(protocol_t *protocol, void *data, size_t len)
```

```
{
    if (!protocol->transport->is_audio_channel_open
|| !protocol->transport->send_audio)
    {
        ESP_LOGE(TAG, "Send audio not supported");
        return;
    }

    if
(protocol->transport->is_audio_channel_open(protocol->transport))
    {
        protocol->transport->send_audio(protocol->transport, data,
len);
    }
    else
    {
        ESP_LOGE(TAG, "Audio channel not open");
    }
}

void protocol_send_hello(protocol_t *protocol)
{
    char *text = NULL;
    char *transport = NULL;

    if (!protocol_can_send_text(protocol))
    {
        return;
    }

    switch (protocol->transport->type)
    {
    case PROTOCOL_TRANSPORT_TYPE_WEBSOCKET:
        transport = "websocket";
        break;

    default:
        transport = "unknown";
        break;
    }
    asprintf(&text, "{\"audio_params\": {\"channels\": %d, \"format\":
\"opus\", \"frame_duration\": 60, \"sample_rate\": %d}, \"transport\":
\"%s\", \"type\": \"hello\", \"version\": 1}",
             BSP_CODEC_CHANNEL_COUNT,
```

```
            BSP_CODEC_SAMPLE_RATE,
            transport);
    protocol->transport->send_text(protocol->transport, text);
    free(text);

    // 等待服务器回复
    xEventGroupWaitBits(protocol->event_group, PROTOCOL_EVENT_HELLO,
pdTRUE, pdTRUE, portMAX_DELAY);
}

void protocol_send_wake_word(protocol_t *protocol, char *word)
{
    char *text = NULL;
    if (!protocol_can_send_text(protocol))
    {
        return;
    }

    asprintf(&text, "{\"session_id\": \"%s\", \"state\": \"detect\",
\"text\": \"%s\", \"type\": \"listen\"}",
            protocol->session_id ? protocol->session_id : "",
            word);
    protocol->transport->send_text(protocol->transport, text);
    free(text);
}

void protocol_send_start_listening(protocol_t *protocol)
{
    char *text = NULL;
    if (!protocol_can_send_text(protocol))
    {
        return;
    }
    asprintf(&text, "{\"mode\": \"manual\", \"session_id\": \"%s\",
\"state\": \"start\", \"type\": \"listen\"}",
            protocol->session_id ? protocol->session_id : "");
    protocol->transport->send_text(protocol->transport, text);
    free(text);
}

void protocol_send_stop_listening(protocol_t *protocol)
{
    char *text = NULL;
    if (!protocol_can_send_text(protocol))
```

```
    {
        return;
    }
    asprintf(&text, "{\"session_id\": \"%s\", \"state\": \"stop\",
\"type\": \"listen\"}",
             protocol->session_id ? protocol->session_id : "");
    protocol->transport->send_text(protocol->transport, text);
    free(text);
}

void protocol_abort_speaking(protocol_t *protocol,
protocol_abort_reason_t reason)
{
    char *text = NULL;
    if (!protocol_can_send_text(protocol))
    {
        return;
    }

    switch (reason)
    {
    case PROTOCOL_ABORT_REASON_WAKEUP:
        asprintf(&text, "{\"reason\": \"wake_word_detected\",
\"session_id\": \"%s\", \"type\": \"abort\"}",
                 protocol->session_id ? protocol->session_id : "");
        break;

    default:
        asprintf(&text, "{\"session_id\": \"%s\", \"type\":
\"abort\"}",
                 protocol->session_id ? protocol->session_id : "");
        break;
    }

    protocol->transport->send_text(protocol->transport, text);
    free(text);
}

void protocol_send_iot_descriptor(protocol_t *protocol, cJSON
*descriptor_json)
{
    if (!protocol_can_send_text(protocol))
    {
        cJSON_Delete(descriptor_json);
```

```
        return;
    }

    cJSON *root = cJSON_CreateObject();
    cJSON_AddItemToObject(root, "descriptors", descriptor_json);
    cJSON_AddStringToObject(root, "type", "iot");
    cJSON_AddStringToObject(root, "session_id", protocol->session_id);
    cJSON_AddBoolToObject(root, "update", true);
    char *json_str = cJSON_PrintUnformatted(root);
    cJSON_Delete(root);

    protocol->transport->send_text(protocol->transport, json_str);
    free(json_str);
}

void protocol_send_iot_state(protocol_t *protocol, cJSON *state_json)
{
    if (!protocol_can_send_text(protocol))
    {
        cJSON_Delete(state_json);
        return;
    }

    cJSON *root = cJSON_CreateObject();
    cJSON_AddItemToObject(root, "states", state_json);
    cJSON_AddStringToObject(root, "type", "iot");
    cJSON_AddStringToObject(root, "session_id", protocol->session_id);
    cJSON_AddBoolToObject(root, "update", true);
    char *json_str = cJSON_PrintUnformatted(root);
    cJSON_Delete(root);

    protocol->transport->send_text(protocol->transport, json_str);
    free(json_str);
}



// callback 系列函数，主要负责处理来自于服务器的命令所产生的回调
void protocol_set_stt_callback(protocol_t *protocol, text_callback_t
callback)
{
    protocol->stt_callback = callback;
}
```

```
void protocol_set_llm_callback(protocol_t *protocol, text_callback_t
callback)
{
    protocol->llm_callback = callback;
}

void protocol_set_tts_start_callback(protocol_t *protocol,
action_callback_t callback)
{
    protocol->tts_start_callback = callback;
}

void protocol_set_tts_stop_callback(protocol_t *protocol,
action_callback_t callback)
{
    protocol->tts_stop_callback = callback;
}

void protocol_set_tts_sentence_start_callback(protocol_t *protocol,
text_callback_t callback)
{
    protocol->tts_sentence_start_callback = callback;
}

void protocol_set_iot_callback(protocol_t *protocol, text_callback_t
callback)
{
    protocol->iot_callback = callback;
}

void protocol_set_audio_in_callback(protocol_t *protocol,
audio_callback_t callback)
{
    protocol->audio_in_callback = callback;
}

void protocol_set_audio_channel_close_callback(protocol_t *protocol,
action_callback_t callback)
{
    protocol->audio_channel_close_callback = callback;
}
```

3.4.2 协议传输层接口定义

1）protocol_transport.h

```
#ifndef PROTOCOL_TRANSPORT_H
#define PROTOCOL_TRANSPORT_H

#include <esp_types.h>

typedef enum
{
    PROTOCOL_TRANSPORT_TYPE_WEBSOCKET = 0,
    PROTOCOL_TRANSPORT_TYPE_MQTT,
} protocol_transport_type_t;

typedef struct
{
    void (*audio_callback)(void *protocol, void *data, size_t len);
    void (*text_callback)(void *protocol, char *text);
    void (*channel_close_callback)(void *protocol);
} protocol_transport_callbacks_t;

typedef struct protocol_transport
{
    protocol_transport_type_t type;
    void (*start)(struct protocol_transport *transport);
    bool (*open_audio_channel)(struct protocol_transport *transport);
    void (*close_audio_channel)(struct protocol_transport *transport);
    bool (*is_audio_channel_open)(struct protocol_transport
*transport);
    void (*send_audio)(struct protocol_transport *transport, void
*data, size_t len);
    void (*send_text)(struct protocol_transport *transport, char
*text);
    void (*deinit)(struct protocol_transport *transport);
} protocol_transport_t;

protocol_transport_t
*protocol_transport_create_websocket(protocol_transport_callbacks_t
*callbacks, void *protocol);

// protocol_transport_t
*protocol_transport_create_mqtt(protocol_transport_callbacks_t
*callbacks, void *protocol);
#endif
```

3.4.3 协议传输层 websocket 实现

1）protocol_transport_websocket.c

```
#include "protocol_transport.h"
#include <freertos/FreeRTOS.h>
#include <freertos/event_groups.h>
#include <stdlib.h>
#include <string.h>
#include <esp_crt_bundle.h>
#include <esp_log.h>
#include <bsp.h>
#include "esp_websocket_client.h"

#define TAG "[PROTOCOL] Websocket"
#define WEBSOCKET_CONNECTED_BIT (1 << 0)

#define WEBSOCKET_URL "wss://api.tenclass.net/xiaozhi/v1/"
#define WEBSOCKET_TOKEN "test-token"

typedef struct protocol_transport_websocket
{
    protocol_transport_t transport;
    protocol_transport_callbacks_t *callbacks;
    void *protocol;
    EventGroupHandle_t event_group;
    esp_websocket_client_handle_t client;
} protocol_transport_websocket_t;

// 处理回调
static void protocol_transport_websocket_event_handler(void
*handler_args, esp_event_base_t base, int32_t event_id, void
*event_data)
{
    esp_websocket_event_data_t *data = (esp_websocket_event_data_t
*)event_data;
    protocol_transport_websocket_t *transport =
(protocol_transport_websocket_t *)handler_args;
    switch (event_id)
    {
    case WEBSOCKET_EVENT_BEGIN:
        ESP_LOGI(TAG, "WEBSOCKET_EVENT_BEGIN");
        break;
    case WEBSOCKET_EVENT_CONNECTED:
        ESP_LOGI(TAG, "WEBSOCKET_EVENT_CONNECTED");
        xEventGroupSetBits(transport->event_group,
WEBSOCKET_CONNECTED_BIT);
```

```
        break;
    case WEBSOCKET_EVENT_DISCONNECTED:
        ESP_LOGI(TAG, "WEBSOCKET_EVENT_DISCONNECTED");
        break;
    case WEBSOCKET_EVENT_DATA:
        ESP_LOGD(TAG, "WEBSOCKET_EVENT_DATA");
        if (data->op_code == 0x01)
        {
            // 文本数据
            if (transport->callbacks->text_callback)
            {
                char *ptr = data->data_ptr;
                ptr[data->data_len] = 0;
                ESP_LOGI(TAG, "received text: %s", ptr);
                transport->callbacks->text_callback(transport->protocol
, ptr);
            }
        }
        else if (data->op_code == 0x02)
        {
            // 音频数据
            if (transport->callbacks->audio_callback)
            {
                transport->callbacks->audio_callback(transport->protoco
l, (void *)data->data_ptr, data->data_len);
            }
        }

        break;
    case WEBSOCKET_EVENT_ERROR:
        ESP_LOGI(TAG, "WEBSOCKET_EVENT_ERROR");
        break;
    case WEBSOCKET_EVENT_FINISH:
        ESP_LOGI(TAG, "WEBSOCKET_EVENT_FINISH");
        if (transport->callbacks->channel_close_callback)
        {
            transport->callbacks->channel_close_callback(transport->pro
tocol);
        }
        break;
    }
}

// 开，关，查询状态
```

```
static bool
protocol_transport_websocket_open_audio_channel(protocol_transport_t
*transport)
{
    protocol_transport_websocket_t *ws_transport =
(protocol_transport_websocket_t *)transport;
    // 开启 websocket 音频通道
    esp_websocket_client_config_t websocket_config = {
        .uri = WEBSOCKET_URL,
        .buffer_size = 2048,
        .transport = WEBSOCKET_TRANSPORT_OVER_SSL,
        .crt_bundle_attach = esp_crt_bundle_attach,
        .network_timeout_ms = 5000,
        .reconnect_timeout_ms = 5000,
    };

    ws_transport->client =
esp_websocket_client_init(&websocket_config);
    if (!ws_transport->client)
    {
        return false;
    }

    ESP_ERROR_CHECK(esp_websocket_register_events(ws_transport->client,
WEBSOCKET_EVENT_ANY, protocol_transport_websocket_event_handler, (void
*)ws_transport));

    esp_websocket_client_append_header(ws_transport->client,
"Authorization", "Bearer " WEBSOCKET_TOKEN);
    esp_websocket_client_append_header(ws_transport->client, "Protocol-
Version", "1");
    esp_websocket_client_append_header(ws_transport->client, "Device-
Id", bsp_get_mac_address());
    esp_websocket_client_append_header(ws_transport->client, "Client-
Id", bsp_get_uuid());

    if (esp_websocket_client_start(ws_transport->client) != ESP_OK)
    {
        ESP_LOGE(TAG, "Failed to connect to websocket server");
        goto DESTROY_FAIL;
    }

    xEventGroupWaitBits(ws_transport->event_group,
WEBSOCKET_CONNECTED_BIT, pdTRUE, pdTRUE, portMAX_DELAY);
```

```
    return true;

DESTROY_FAIL:
    esp_websocket_client_destroy(ws_transport->client);
    ws_transport->client = NULL;
    return false;
}

static void
protocol_transport_websocket_close_audio_channel(protocol_transport_t
*transport)
{
    protocol_transport_websocket_t *ws_transport =
(protocol_transport_websocket_t *)transport;
    if (esp_websocket_client_is_connected(ws_transport->client))
    {
        esp_websocket_client_close(ws_transport->client,
portMAX_DELAY);
    }
    esp_websocket_client_destroy(ws_transport->client);
    ws_transport->client = NULL;
}

static bool
protocol_transport_websocket_is_audio_channel_opened(protocol_transport
_t *transport)
{
    protocol_transport_websocket_t *ws_transport =
(protocol_transport_websocket_t *)transport;
    return esp_websocket_client_is_connected(ws_transport->client);
}

// 发送二进制和文本
static void
protocol_transport_websocket_send_audio(protocol_transport_t
*transport, void *data, size_t len)
{
    protocol_transport_websocket_t *ws_transport =
(protocol_transport_websocket_t *)transport;

    if (esp_websocket_client_is_connected(ws_transport->client))
    {
```

```
        assert(esp_websocket_client_send_bin(ws_transport->client,
data, len, portMAX_DELAY) > 0);
    }
    else
    {
        ESP_LOGE(TAG, "Websocket not connected");
    }
}

static void protocol_transport_websocket_send_text(protocol_transport_t
*transport, char *text)
{
    protocol_transport_websocket_t *ws_transport =
(protocol_transport_websocket_t *)transport;
    if (esp_websocket_client_is_connected(ws_transport->client))
    {
        ESP_LOGI(TAG, "Sending text %s", text);
        assert(esp_websocket_client_send_text(ws_transport->client,
text, strlen(text), portMAX_DELAY) > 0);
    }
    else
    {
        ESP_LOGE(TAG, "Websocket not connected");
    }
}

// 清理
static void protocol_transport_websocket_deinit(protocol_transport_t
*transport)
{
    protocol_transport_websocket_t *ws_transport =
(protocol_transport_websocket_t *)transport;
    if (ws_transport->client)
    {
        protocol_transport_websocket_close_audio_channel(transport);
    }
    vEventGroupDelete(ws_transport->event_group);
    free(ws_transport);
}

// 初始化
protocol_transport_t
*protocol_transport_create_websocket(protocol_transport_callbacks_t
*callbacks, void *protocol)
```

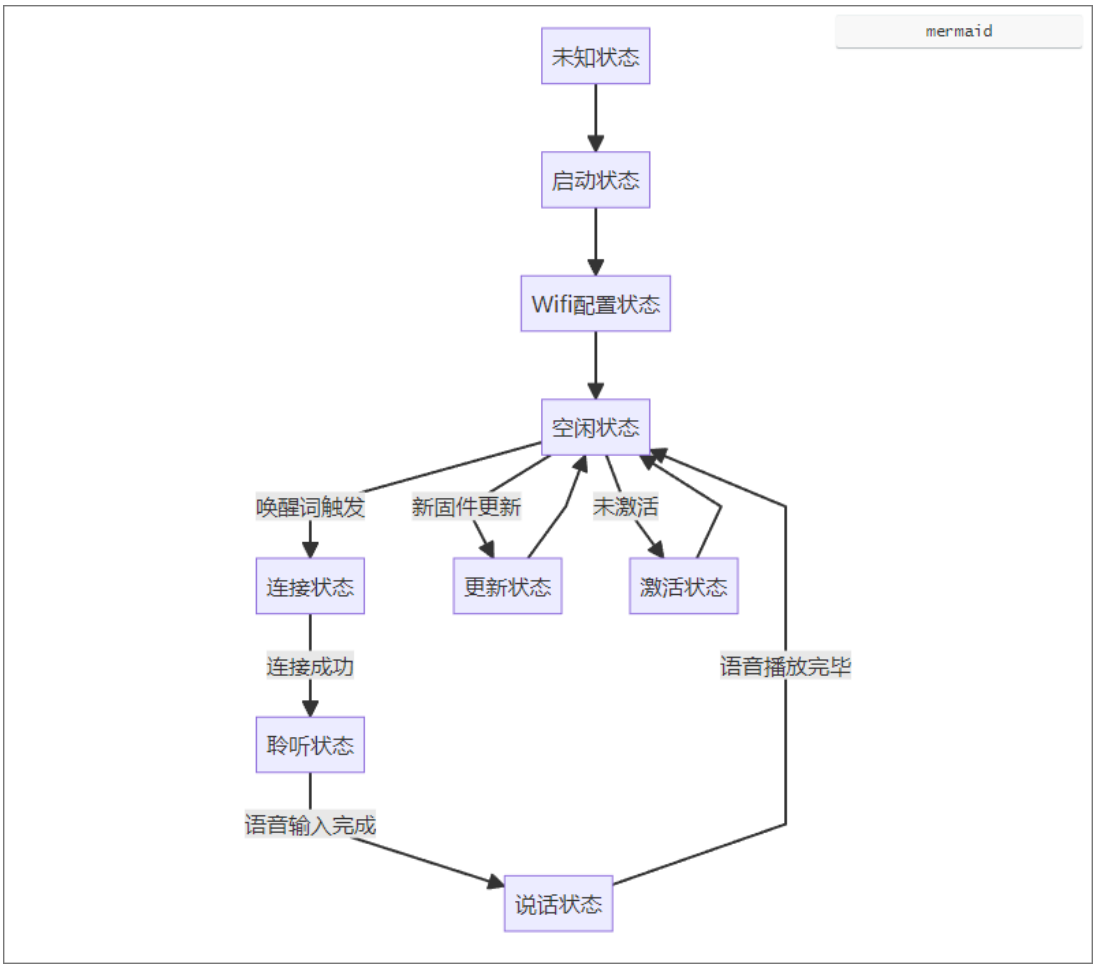
```
{
    protocol_transport_websocket_t *transport =
malloc(sizeof(protocol_transport_websocket_t));
    assert(transport);
    memset(transport, 0, sizeof(protocol_transport_websocket_t));

    transport->transport.type = PROTOCOL_TRANSPORT_TYPE_WEBSOCKET,
    transport->transport.start = NULL,
    transport->transport.open_audio_channel =
protocol_transport_websocket_open_audio_channel,
    transport->transport.close_audio_channel =
protocol_transport_websocket_close_audio_channel,
    transport->transport.is_audio_channel_open =
protocol_transport_websocket_is_audio_channel_opened,
    transport->transport.send_audio =
protocol_transport_websocket_send_audio,
    transport->transport.send_text =
protocol_transport_websocket_send_text,
    transport->transport.deinit = protocol_transport_websocket_deinit,

    transport->protocol = protocol;
    transport->callbacks = callbacks;
    transport->event_group = xEventGroupCreate();
    return (protocol_transport_t *)transport;
}
```

3.5 主控逻辑

主控逻辑需要完成设备的状态切换，如下图所示：



1）application.h

```
#ifndef APPLICATION_H
#define APPLICATION_H

void application_init(void);

#endif
```

2）application.c

```
#include "application.h"
#include "ota.h"
#include "bsp.h"
#include "protocol.h"
#include "audio_processor.h"
#include <stdlib.h>
#include <esp_err.h>
#include <esp_log.h>
#include <assert.h>
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>
```

```
#include <freertos/event_groups.h>
#include "speaker_thing.h"

#define TAG "APP"

#define APPLICATION_LISTENING_EVENT_BIT (1 << 0)

char *state_str[] = {
    "STARTING",
    "IDLE",
    "ACTIVATING",
    "CONNECTING",
    "LISTENING",
    "SPEAKING",
};

typedef enum
{
    DEVICE_STARTING,
    DEVICE_IDLE,
    DEVICE_ACTIVATING,
    DEVICE_CONNECTING,
    DEVICE_LISTENING,
    DEVICE_SPEAKING,
} device_state_t;

typedef struct
{
    protocol_t *protocol;
    device_state_t state;
    thing_list_t *thing_list;

    TaskHandle_t audio_upload_task;
} application_t;

static application_t *s_application = NULL;

static void application_alert(char *msg)
{
    ESP_LOGW(TAG, "%s", msg);
}

static void application_ota()
{
```

```
    ota_t *ota = ota_create();
    assert(ota);
    while (1)
    {
        ota_check_new_version(ota);
        if (!ota->has_activation_code)
        {
            break;
        }
        char *msg = NULL;
        asprintf(&msg, "Activation code: %s", ota->activation_code);
        application_alert(msg);
        free(msg);
        vTaskDelay(pdMS_TO_TICKS(10000));
    }
    ota_free(ota);
}

static void application_set_state(device_state_t state)
{
    if (s_application->state == state)
    {
        return;
    }
    ESP_LOGI(TAG, "State changed: %s -> %s",
state_str[s_application->state], state_str[state]);
    bsp_led_set_state((bsp_led_state_t)state);
    s_application->state = state;
}
static void application_wakenet_handler(void *arg)
{
    switch (s_application->state)
    {
    case DEVICE_IDLE:
        application_set_state(DEVICE_CONNECTING);
        if (!protocol_is_audio_channel_open(s_application->protocol))
        {
            protocol_open_audio_channel(s_application->protocol);
        }
        protocol_send_hello(s_application->protocol);
        // 发 IOT 描述和状态 JSON
        protocol_send_iot_descriptor(s_application->protocol,
thing_list_get_descriptor_json(s_application->thing_list));
```

```
        protocol_send_iot_state(s_application->protocol,
thing_list_get_state_json(s_application->thing_list));
        break;
    case DEVICE_SPEAKING:
        protocol_abort_speaking(s_application->protocol,
PROTOCOL_ABORT_REASON_WAKEUP);
        break;
    default:
        break;
    }
    application_set_state(DEVICE_IDLE);
    protocol_send_wake_word(s_application->protocol, "你好小智");
}

static void application_audio_upload_task(void *arg)
{
    while (1)
    {
        size_t size = 0;
        void *buf = audio_processor_read(&size);
        if (size > 0)
        {
            if (s_application->state == DEVICE_LISTENING)
            {
                ESP_LOGD(TAG, "Uploading audio... %u", size);
                protocol_send_audio(s_application->protocol, buf,
size);
            }

            free(buf);
        }
    }
}

static void application_vad_handler(void *arg, vad_state_t vad_state)
{
    if (vad_state == VAD_SPEECH && s_application->state == DEVICE_IDLE)
    {
        protocol_send_start_listening(s_application->protocol);
        application_set_state(DEVICE_LISTENING);
    }
    else if (vad_state == VAD_SILENCE && s_application->state ==
DEVICE_LISTENING)
    {
```

```
        protocol_send_stop_listening(s_application->protocol);
        application_set_state(DEVICE_IDLE);
    }
}

static void application_llm_handler(char *text)
{
    char *msg = NULL;
    asprintf(&msg, "LLM: %s", text);
    application_alert(msg);
    free(msg);
}

static void application_stt_handler(char *text)
{
    char *msg = NULL;
    asprintf(&msg, "User speaking: %s", text);
    application_alert(msg);
    free(msg);
}

static void application_iot_handler(char *text)
{
    thing_list_invoke(s_application->thing_list, (cJSON *)text);
}

static void application_tts_sentence_start_handler(char *text)
{
    char *msg = NULL;
    asprintf(&msg, "Xiaozhi speaking: %s", text);
    application_alert(msg);
    free(msg);
}

static void application_tts_start_handler(void)
{
    application_set_state(DEVICE_SPEAKING);
}

static void application_tts_stop_handler(void)
{
    application_set_state(DEVICE_IDLE);
}
```

```
static void application_audio_in_handler(void *buf, size_t len)
{
    ESP_LOGD(TAG, "Audio in: %u", len);
    audio_processor_write(buf, len);
}

static void application_audio_channel_close_handler(void)
{
    application_set_state(DEVICE_IDLE);
    audio_processor_reset_wakenet();
}

void application_init(void)
{
    s_application = malloc(sizeof(application_t));
    assert(s_application);
    s_application->state = DEVICE_STARTING;

    bsp_init();
    application_ota();

    // 获取 thing_list 并添加 thing
    s_application->thing_list = thing_list_new();
    thing_list_add(s_application->thing_list, speaker_thing_create());

    // 初始化音频处理模块
    audio_processor_init();
    audio_processor_set_wakenet_callback(application_wakenet_handler,
NULL);
    audio_processor_set_vad_callback(application_vad_handler, NULL);

    // 初始化协议模块
    s_application->protocol =
protocol_create(PROTOCOL_TRANSPORT_TYPE_WEBSOCKET);
    protocol_set_audio_channel_close_callback(s_application->protocol,
application_audio_channel_close_handler);
    protocol_set_audio_in_callback(s_application->protocol,
application_audio_in_handler);
    protocol_set_llm_callback(s_application->protocol,
application_llm_handler);
    protocol_set_stt_callback(s_application->protocol,
application_stt_handler);
    protocol_set_tts_sentence_start_callback(s_application->protocol,
application_tts_sentence_start_handler);
```

```
    protocol_set_tts_start_callback(s_application->protocol,
application_tts_start_handler);
    protocol_set_tts_stop_callback(s_application->protocol,
application_tts_stop_handler);
    protocol_set_iot_callback(s_application->protocol,
application_iot_handler);

    xTaskCreate(application_audio_upload_task, "audio_upload_task",
4096, NULL, 5, &s_application->audio_upload_task);

    audio_processor_start();
    application_set_state(DEVICE_IDLE);
}
```

3.6 IOT 控制接口

3.6.1 自定义接口

1）mylist.h

```
#ifndef MYLIST_H
#define MYLIST_H

#include <esp_types.h>
#include <esp_heap_caps.h>
#include <stdlib.h>
#include <string.h>

typedef struct
{
    void **obj_p;
    size_t size;
    size_t capacity;
} general_list_t;

static inline void *object_new(size_t size)
{
    void *ret = heap_caps_malloc(size, MALLOC_CAP_SPIRAM);
    if (ret)
    {
        memset(ret, 0, size);
    }
    return ret;
}
```

```
#define general_list_new() (general_list_t
*)object_new(sizeof(general_list_t))

static inline bool general_list_add(general_list_t *list, void *obj)
{
    if (!list)
    {
        return false;
    }

    if (list->size >= list->capacity)
    {
        // 扩容操作
        size_t new_capcity = list->capacity ? list->capacity * 2 : 1;
        void **new_obj_p = (void **)heap_caps_realloc(list->obj_p,
new_capcity * sizeof(void *), MALLOC_CAP_SPIRAM);
        if (!new_obj_p)
        {
            return false;
        }
        list->obj_p = new_obj_p;
        list->capacity = new_capcity;
    }

    list->obj_p[list->size++] = obj;
    return true;
}

#define foreach_in_list(type, item, list)      \
    for (size_t _i = 0; _i < list->size; _i++) \
        for (type item = list->obj_p[_i]; item; item = NULL)

#endif
```

3.6.2 属性（参数）

1）properties.h

```
#ifndef PROPERTIES_H
#define PROPERTIES_H

#include <esp_types.h>
#include <cJSON.h>
#include "mylist.h"

typedef enum
```

```
{
    PROPERTY_VALUE_TYPE_NUMBER,
    PROPERTY_VALUE_TYPE_BOOL,
    PROPERTY_VALUE_TYPE_STRING,
} property_value_type_t;

typedef union
{
    int number;
    bool boolean;
    char *str;
} property_value_t;

typedef struct
{
    char *name;
    char *description;
    property_value_type_t type;
    property_value_t value;
} property_t;

#define property_list_t general_list_t
#define property_list_add general_list_add
#define property_list_new general_list_new

property_t *property_create(char *name, char *description,
property_value_type_t type, property_value_t value);

property_t *property_list_get(property_list_t *list, char *name);
cJSON *property_list_get_descriptor_json(property_list_t *list);
cJSON *property_list_get_state_json(property_list_t *list);

#endif
```

2）properties.c

```
#include "properties.h"

property_t *property_create(char *name, char *description,
property_value_type_t type, property_value_t value)
{
    property_t *property = (property_t
*)object_new(sizeof(property_t));
    assert(property);
    property->name = strdup(name);
    property->description = strdup(description);
```

```
    property->type = type;
    if (type == PROPERTY_VALUE_TYPE_STRING)
    {
        property->value.str = strdup(value.str);
    }
    else
    {
        property->value = value;
    }

    return property;
}

property_t *property_list_get(property_list_t *list, char *name)
{
    foreach_in_list(property_t *, property, list)
    {
        if (strcmp(property->name, name) == 0)
        {
            return property;
        }
    }
    return NULL;
}

cJSON *property_list_get_descriptor_json(property_list_t *list)
{
    cJSON *root = cJSON_CreateObject();
    assert(root);
    foreach_in_list(property_t *, property, list)
    {
        cJSON *prop = cJSON_CreateObject();
        assert(prop);
        cJSON_AddStringToObject(prop, "description",
property->description);
        switch (property->type)
        {
        case PROPERTY_VALUE_TYPE_BOOL:
            cJSON_AddStringToObject(prop, "type", "bool");
            break;
        case PROPERTY_VALUE_TYPE_NUMBER:
            cJSON_AddStringToObject(prop, "type", "number");
            break;
        case PROPERTY_VALUE_TYPE_STRING:
```

```
            cJSON_AddStringToObject(prop, "type", "string");
            break;
        default:
            break;
        }
        cJSON_AddItemToObject(root, property->name, prop);
    }
    return root;
}

cJSON *property_list_get_state_json(property_list_t *list)
{
    cJSON *root = cJSON_CreateObject();
    assert(root);
    foreach_in_list(property_t *, property, list)
    {
        switch (property->type)
        {
        case PROPERTY_VALUE_TYPE_BOOL:
            cJSON_AddBoolToObject(root, property->name,
property->value.boolean);
            break;
        case PROPERTY_VALUE_TYPE_NUMBER:
            cJSON_AddNumberToObject(root, property->name,
property->value.number);
            break;
        case PROPERTY_VALUE_TYPE_STRING:
            cJSON_AddStringToObject(root, property->name,
strdup(property->value.str));
        }
    }
    return root;
}
```

3.6.3 方法（函数）

1）methods.h

```
#ifndef METHOD_H
#define METHOD_H

#include "properties.h"

typedef struct
{
```

```
    char *name;
    char *description;
    property_list_t *parameters;
    void (*callback)(void *context, property_list_t *parameters);
    void *context;
} method_t;

#define method_list_t general_list_t
#define method_list_add general_list_add
#define method_list_new general_list_new

method_t *method_create(char *name, char *description, property_list_t
*parameters, void (*callback)(void *, property_list_t *), void
*context);
method_t *method_list_get(method_list_t *list, char *name);
cJSON *method_list_get_descriptor_json(method_list_t *list);

#endif
```

2）methods.c

```
#include "methods.h"

method_t *method_create(char *name, char *description, property_list_t
*parameters, void (*callback)(void *, property_list_t *), void
*context)
{
    method_t *method = (method_t *)object_new(sizeof(method_t));
    if (!method)
    {
        return NULL;
    }

    method->name = strdup(name);
    method->description = strdup(description);
    method->parameters = parameters;
    method->callback = callback;
    method->context = context;

    return method;
}

method_t *method_list_get(method_list_t *list, char *name)
{
    foreach_in_list(method_t *, method, list)
    {
```

```
        if (strcmp(method->name, name) == 0)
        {
            return method;
        }
    }
    return NULL;
}

cJSON *method_list_get_descriptor_json(method_list_t *list)
{
    cJSON *root = cJSON_CreateObject();
    assert(root);

    foreach_in_list(method_t *, method, list)
    {
        cJSON *method_json = cJSON_CreateObject();
        assert(method_json);
        cJSON_AddStringToObject(method_json, "description",
method->description);
        cJSON_AddItemToObject(method_json, "parameters",
property_list_get_descriptor_json(method->parameters));

        cJSON_AddItemToObject(root, method->name, method_json);
    }
    return root;
}
```

3.6.4 IOT 设备

1）things.h

```
#ifndef THINGS_H
#define THINGS_H

#include "methods.h"

typedef struct
{
    char *description;
    char *name;
    method_list_t *methods;
    property_list_t *properties;
} thing_t;

#define thing_list_t general_list_t
#define thing_list_add general_list_add
```

```
#define thing_list_new general_list_new

thing_t *thing_create(char *name, char *description, method_list_t
*methods, property_list_t *properties);

cJSON *thing_get_descriptor_json(thing_t *thing);
cJSON *thing_get_state_json(thing_t *thing);

cJSON *thing_list_get_descriptor_json(thing_list_t *list);
cJSON *thing_list_get_state_json(thing_list_t *list);

// 从 Thinglist 中获取指定名字的 thing
thing_t *thing_list_get(thing_list_t *list, char *name);

// 调用 thing 执行 command
void thing_invoke(thing_t *thing, cJSON *command);

// 调用 thing_list 执行 command 列表
void thing_list_invoke(thing_list_t *list, cJSON *command_array);

#endif
```

2）things.c

```
#include "things.h"
#include <esp_log.h>

#define TAG "things"

thing_t *thing_create(char *name, char *description, method_list_t
*methods, property_list_t *properties)
{
    thing_t *thing = (thing_t *)object_new(sizeof(thing_t));
    assert(thing);

    thing->name = strdup(name);
    thing->description = strdup(description);
    thing->methods = methods;
    thing->properties = properties;
    return thing;
}

cJSON *thing_get_descriptor_json(thing_t *thing)
{
    cJSON *root = cJSON_CreateObject();
    assert(root);
```

```
    cJSON_AddStringToObject(root, "name", thing->name);
    cJSON_AddStringToObject(root, "description", thing->description);
    cJSON_AddItemToObject(root, "methods",
method_list_get_descriptor_json(thing->methods));
    cJSON_AddItemToObject(root, "properties",
property_list_get_descriptor_json(thing->properties));
    return root;
}

cJSON *thing_get_state_json(thing_t *thing)
{
    cJSON *root = cJSON_CreateObject();
    cJSON_AddStringToObject(root, "name", thing->name);
    cJSON_AddItemToObject(root, "state",
property_list_get_state_json(thing->properties));
    return root;
}

cJSON *thing_list_get_descriptor_json(thing_list_t *list)
{
    cJSON *root = cJSON_CreateArray();
    assert(root);
    foreach_in_list(thing_t *, thing, list)
    {
        cJSON_AddItemToArray(root, thing_get_descriptor_json(thing));
    }
    return root;
}

cJSON *thing_list_get_state_json(thing_list_t *list)
{
    cJSON *root = cJSON_CreateArray();
    assert(root);
    foreach_in_list(thing_t *, thing, list)
    {
        cJSON_AddItemToArray(root, thing_get_state_json(thing));
    }
    return root;
}

thing_t *thing_list_get(thing_list_t *list, char *name)
{
    foreach_in_list(thing_t *, thing, list)
    {
```

```
        // 挨个比较名字，匹配返回
        if (strcmp(thing->name, name) == 0)
        {
            return thing;
        }
    }
    // 找不到匹配项返回 NULL
    return NULL;
}

void thing_invoke(thing_t *thing, cJSON *command)
{
    cJSON *method_name = cJSON_GetObjectItem(command, "method");
    if (cJSON_IsString(method_name))
    {
        // 根据 method 名字，获取 method
        method_t *method = method_list_get(thing->methods,
method_name->valuestring);

        // 解析参数的值，传入 method_parameters，并调用 method 回调
        cJSON *method_parameters = cJSON_GetObjectItem(command,
"parameters");
        if (!method_parameters)
        {
            ESP_LOGW(TAG, "method_parameters is null");
            return;
        }

        // 遍历 method 的参数列表，匹配每一个值
        foreach_in_list(property_t *, param, method->parameters)
        {
            // 根据函数的参数名称，获取实际的参数值
            cJSON *param_json = cJSON_GetObjectItem(method_parameters,
param->name);
            if (!param_json)
            {
                ESP_LOGW(TAG, "Requried parameter is null");
                return;
            }

            // 根据参数类型，解析传入参数值
            switch (param->type)
            {
            case PROPERTY_VALUE_TYPE_BOOL:
```

```
                param->value.boolean = cJSON_IsTrue(param_json);
                break;
            case PROPERTY_VALUE_TYPE_NUMBER:
                param->value.number = param_json->valueint;
                break;
            case PROPERTY_VALUE_TYPE_STRING:
                param->value.str = strdup(param_json->valuestring);
                break;
            default:
                break;
            }
        }
        method->callback(method->context, method->parameters);

        // 释放 strdup 分配的内存
        foreach_in_list(property_t *, param, method->parameters)
        {
            if (param->type == PROPERTY_VALUE_TYPE_STRING &&
param->value.str)
            {
                free(param->value.str);
                param->value.str = NULL;
            }
        }
    }
}

void thing_list_invoke(thing_list_t *list, cJSON *command_array)
{
    for (size_t i = 0; i < cJSON_GetArraySize(command_array); i++)
    {
        cJSON *command = cJSON_GetArrayItem(command_array, i);

        // 找到要设置的 thing 是哪个
        cJSON *thing_name = cJSON_GetObjectItemCaseSensitive(command,
"name");
        if (cJSON_IsString(thing_name))
        {
            // 按照名字找到 thing
            thing_t *thing = thing_list_get(list,
thing_name->valuestring);

            // 找到后执行命，找不到打印错误日志
            if (thing)
```

```
            {
                thing_invoke(thing, command);
            }
            else
            {
                ESP_LOGW(TAG, "thing %s not found",
thing_name->valuestring);
            }
        }
    }
}
```

3.7 其他

1）growable_buffer.h

```
#if !defined(__GROUABLE_BUFFER_H__)
#define __GROUABLE_BUFFER_H__

#include <esp_types.h>

typedef struct growable_buffer *growable_buffer_t;

growable_buffer_t growable_buffer_create(void);

void growable_buffer_destroy(growable_buffer_t buffer);

void growable_buffer_add(growable_buffer_t buffer, void *data, size_t
len);

char* growable_buffer_get_content(growable_buffer_t buffer);

void growable_buffer_reset(growable_buffer_t buffer);


#endif // __GROUABLE_BUFFER_H__
```

2）growable_buffer.c

```
#include "growable_buffer.h"
#include "object.h"

struct growable_buffer
{
    char *buffer; // 动态分配的缓冲区
```

```
    size_t size;  // 当前缓冲区大小
};

growable_buffer_t growable_buffer_create(void)
{
    growable_buffer_t buffer = object_create(sizeof(struct
growable_buffer));
    buffer->buffer = malloc(1);
    buffer->size = 0;
    buffer->buffer[0] = 0;
    return buffer;
}

void growable_buffer_destroy(growable_buffer_t buffer)
{
    if (buffer == NULL)
        return;
    free(buffer->buffer);
    free(buffer);
}

void growable_buffer_add(growable_buffer_t buffer, void *data, size_t
len)
{
    if (buffer == NULL)
        return;
    if (data == NULL || len == 0)
        return;
    size_t new_size = buffer->size + len;
    char *new_buffer = realloc(buffer->buffer, new_size + 1);
    if (new_buffer == NULL)
        return;
    buffer->buffer = new_buffer;
    memcpy(buffer->buffer + buffer->size, data, len);
    buffer->size = new_size;
    buffer->buffer[buffer->size] = 0; // 确保字符串以空字符结尾
}

char *growable_buffer_get_content(growable_buffer_t buffer){
    return buffer->buffer;
}

void growable_buffer_reset(growable_buffer_t buffer){
    if (buffer == NULL)
```

```
        return;
    buffer->buffer = realloc(buffer->buffer, 1);
    buffer->size = 0;
    buffer->buffer[0] = 0;
}
```

3）nvs_settings.h

```
#ifndef NVS_SETTINGS_H
#define NVS_SETTINGS_H

typedef struct settings settings_t;

settings_t* nvs_settings_create(void);

char* nvs_settings_get_string(settings_t *settings, const char *key);

void nvs_settings_set_string(settings_t *settings, const char *key,
const char *value);

void nvs_settings_free(settings_t *settings);

#endif
```

4）nvs_settings.c

```
#include "nvs_settings.h"
#include "nvs_flash.h"

struct settings
{
    nvs_handle_t nvs_flash;
};

settings_t *nvs_settings_create(void)
{
    settings_t *settings = malloc(sizeof(settings_t));
    assert(settings);

    ESP_ERROR_CHECK(nvs_open("settings", NVS_READWRITE,
&settings->nvs_flash));

    return settings;
}

char *nvs_settings_get_string(settings_t *settings, const char *key)
{
```

```
    // 查询需要获取的数据有多长
    size_t required_size = 0;
    int ret = nvs_get_str(settings->nvs_flash, key, NULL,
&required_size);
    if (ret == ESP_ERR_NVS_NOT_FOUND)
    {
        return NULL;
    }
    ESP_ERROR_CHECK(ret);
    char *output = (char *)malloc(required_size);
    assert(output);
    ESP_ERROR_CHECK(nvs_get_str(settings->nvs_flash, key, output,
&required_size));

    return output;
}

void nvs_settings_set_string(settings_t *settings, const char *key,
const char *value)
{
    ESP_ERROR_CHECK(nvs_set_str(settings->nvs_flash, key, value));
    ESP_ERROR_CHECK(nvs_commit(settings->nvs_flash));
}

void nvs_settings_free(settings_t *settings)
{
    nvs_close(settings->nvs_flash);
    free(settings);
}
```

5）ota.h

```
#ifndef OTA_H
#define OTA_H

#include <esp_types.h>

#define OTA_URL "https://api.tenclass.net/xiaozhi/ota/"

typedef struct {
    bool has_activation_code;
    char *activation_code;
    char *message;
    char *chanllege;
} ota_t;
```

```
ota_t *ota_create();

void ota_free(ota_t *ota);

void ota_check_new_version(ota_t *ota);

#endif
```

6）ota.c

```
#include "ota.h"
#include <stdlib.h>
#include <assert.h>
#include <string.h>
#include <esp_http_client.h>
#include <esp_crt_bundle.h>
#include <bsp.h>
#include <esp_log.h>
#include <cJSON.h>
#include "mutable_buffer.h"


#define TAG "OTA"

static esp_err_t _http_event_handler(esp_http_client_event_t *evt)
{
    mutable_buffer_t *buffer = evt->user_data;
    switch (evt->event_id)
    {
    case HTTP_EVENT_ERROR:
        ESP_LOGD(TAG, "HTTP_EVENT_ERROR");
        break;
    case HTTP_EVENT_ON_CONNECTED:
        ESP_LOGD(TAG, "HTTP_EVENT_ON_CONNECTED");
        break;
    case HTTP_EVENT_HEADER_SENT:
        ESP_LOGD(TAG, "HTTP_EVENT_HEADER_SENT");
        break;
    case HTTP_EVENT_ON_HEADER:
        ESP_LOGD(TAG, "HTTP_EVENT_ON_HEADER, key=%s, value=%s",
evt->header_key, evt->header_value);
        break;
    case HTTP_EVENT_ON_DATA:
        ESP_LOGD(TAG, "HTTP_EVENT_ON_DATA, len=%d", evt->data_len);
        // 可能需要数据拼接
```

```
            mutable_buffer_append(buffer, evt->data, evt->data_len);
            break;
        case HTTP_EVENT_ON_FINISH:
            ESP_LOGI(TAG, "Response received: %s",
mutable_buffer_get_buffer(buffer));
            break;
        case HTTP_EVENT_DISCONNECTED:
            ESP_LOGI(TAG, "HTTP_EVENT_DISCONNECTED");
            break;
        case HTTP_EVENT_REDIRECT:
            ESP_LOGD(TAG, "HTTP_EVENT_REDIRECT");
            break;
        }
        return ESP_OK;
}

ota_t *ota_create()
{
    ota_t *ota = malloc(sizeof(ota_t));
    assert(ota);
    memset(ota, 0, sizeof(ota_t));
    return ota;
}

void ota_free(ota_t *ota)
{
    if (ota->activation_code)
    {
        free(ota->activation_code);
    }

    if (ota->chanllege)
    {
        free(ota->chanllege);
    }

    if (ota->message)
    {
        free(ota->message);
    }

    free(ota);
}
```

```
void ota_check_new_version(ota_t *ota)
{
    // 发一个 post http 请求
    mutable_buffer_t *buffer = mutable_buffer_create();
    esp_http_client_config_t config = {
        .url = OTA_URL,
        .method = HTTP_METHOD_POST,
        .event_handler = _http_event_handler,
        .user_data = buffer,
        .crt_bundle_attach = esp_crt_bundle_attach,
    };

    // 创建一个 http 客户端
    esp_http_client_handle_t client = esp_http_client_init(&config);
    assert(client);

    // 添加 Header
    esp_http_client_set_header(client, "Content-Type",
"application/json");
    esp_http_client_set_header(client, "Accept", "application/json");
    esp_http_client_set_header(client, "Client-Id", bsp_get_uuid());
    esp_http_client_set_header(client, "Device-Id",
bsp_get_mac_address());
    esp_http_client_set_header(client, "Accept-Language", "zh-CN");
    esp_http_client_set_header(client, "User-Agent", "bread-
compact_wifi/1.0.0");

    // 添加 post body
    char *bsp_json = bsp_get_json();
    esp_http_client_set_post_field(client, bsp_json, strlen(bsp_json));

    // 发送请求
    esp_http_client_perform(client);
    free(bsp_json);

    // 解析激活码部分
    if (esp_http_client_get_status_code(client) != 200)
    {
        ESP_LOGE(TAG, "Failed to send request");
        goto END;
    }

    cJSON* root = cJSON_Parse(mutable_buffer_get_buffer(buffer));
    if (!root)
```

```
    {
        ESP_LOGE(TAG, "Failed to parse JSON");
        goto END;
    }


    cJSON* activation_obj = cJSON_GetObjectItem(root, "activation");
    if (activation_obj)
    {
        ota->has_activation_code = true;
        cJSON* activation_code = cJSON_GetObjectItem(activation_obj,
"code");
        assert(cJSON_IsString(activation_code));
        cJSON* challenge = cJSON_GetObjectItem(activation_obj,
"challenge");
        assert(cJSON_IsString(challenge));
        cJSON* message = cJSON_GetObjectItem(activation_obj,
"message");
        assert(cJSON_IsString(message));

        ota->activation_code = strdup(activation_code->valuestring);
        ota->chanllege = strdup(challenge->valuestring);
        ota->message = strdup(message->valuestring);
    }
    cJSON_Delete(root);

END:
    // 清理客户端
    mutable_buffer_free(buffer);
    esp_http_client_cleanup(client);
}
```

7）Kconfig.projbuild

```
menu "xiaozhi assistant"

config OTA_VERSION_URL
    string "OTA Version URL"
    default "https://api.tenclass.net/xiaozhi/ota/"
    help
        The application will access this URL to check for updates.

config WEBSOCKET_URL
    string "Websocket URL"
    default "wss://api.tenclass.net/xiaozhi/v1/"
    help
```

```
        Communication with the server through websocket after wake up.

config WEBSOCKET_ACCESS_TOKEN
    string "Websocket Access Token"
    default "test-token"
    help
        Access token for websocket communication.

endmenu
```

8）CMakeLists.txt

```
idf_component_register(SRCS
                       "application.c"

                       "bsp.c"
                       "growable_buffer.c"
                       "main.c"
                       "nvs_settings.c"
                       "ota.c"
                       "system_info.c"
                       "protocol/protocol.c"
                       "protocol/websocket_protocol.c"
                       "things/methods.c"
                       "things/properties.c"
                       "things/things.c"
                   INCLUDE_DIRS "."
                       "protocol"
                       "things"
                              )
```